BIT 4107 – Mobile Application Development

ENHANCED PROFESSIONAL STUDENT LOGBOOK

Student Name: CELESTINE GITAU

Admission Number: BIT/2024/54146

School/Faculty: COMPUTER AND INFORMATICS

Programme: MOBILE APPLICATION DEVELOPMENT

Academic Year:2026

Semester: MAY/AUG

Lecturer: NYORO MICHAEL

Phone Number: 0769722238

Instructions

This logbook is used to document weekly learning activities, practical exercises, skills acquired, challenges encountered, solutions applied, and reflections. Students should complete all sections every week.

Each week to follow remarks in week 1 section.

Autogenerated Table of content

Week 1: Introduction to Mobile Application Development

Practical Activities Undertaken

1. Installed Android Studio and Java Development Kit (JDK).
2. Configured Android SDK and Gradle.
3. Created an Android Virtual Device (AVD) Emulator.
4. Created a new Android Studio project.
5. Explored Android Studio interface (Project Explorer, Code Editor, Layout Editor).
6. Executed the first Android application on the emulator.
7. Studied Android application architecture and project structure.
8. Tested successful project build and execution.

Screenshots to Attach

Android Studio Installation Completed

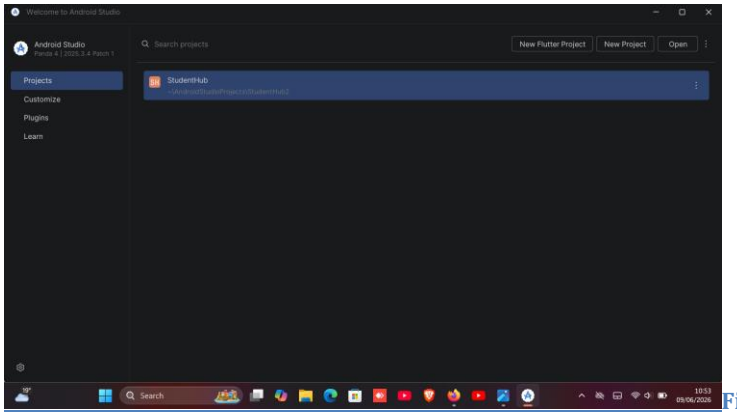


Figure 1.installation process

Created project structure

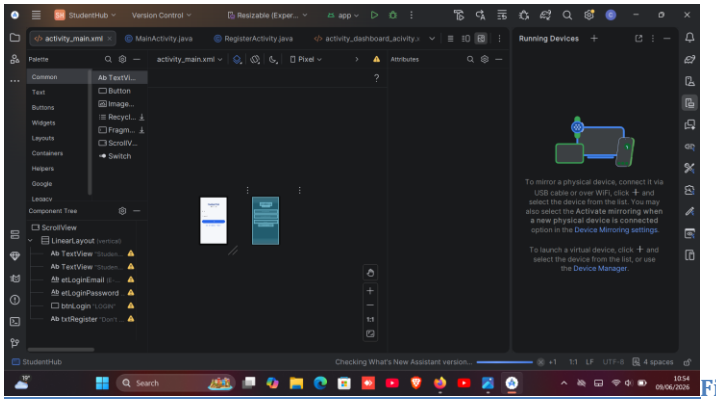
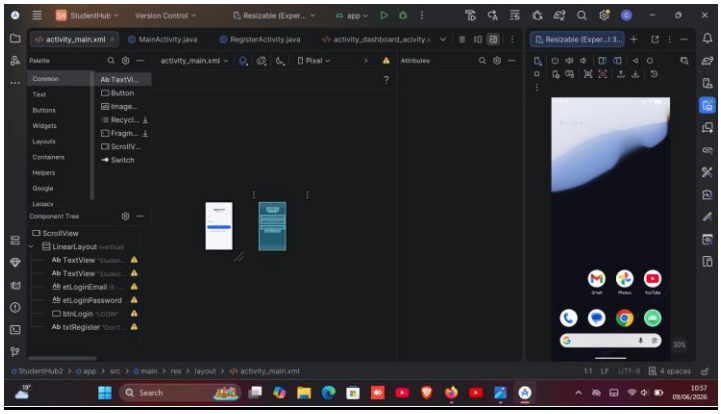


Figure 2.studenthub creation

Emulator working



Tools/Software Used

1. Android Studio
2. Java JDK
3. Android SDK
4. Android Emulator
5. Windows Operating System

Personal Reflection

This week I successfully set up the Android development environment and understood the structure of Android projects. Running my first application on the emulator increased my confidence in mobile application development.

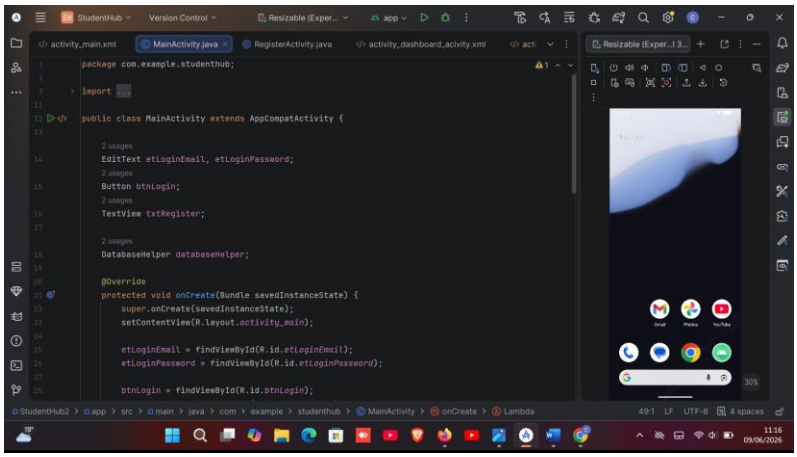
Week 2: Mobile Development Environment

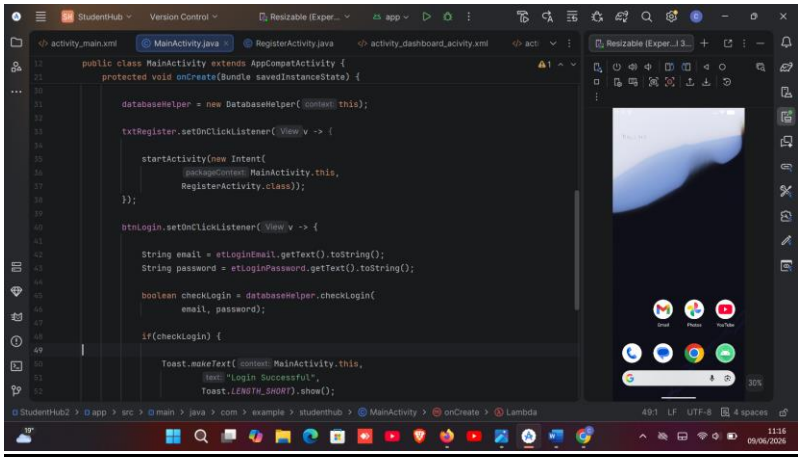
Practical Activities Undertaken

1. Learned Java syntax used in Android development.
2. Created Java classes and methods.
3. Designed simple user interface components.
4. Added Buttons, Text Views and Edit Texts.
5. Developed navigation between application screens.
6. Created the Student Management App project structure.
7. Designed Login Screen and Student Registration Screen.
8. Tested application functionality using Android Emulator.

Screenshots to Attach

Mainactivity.java





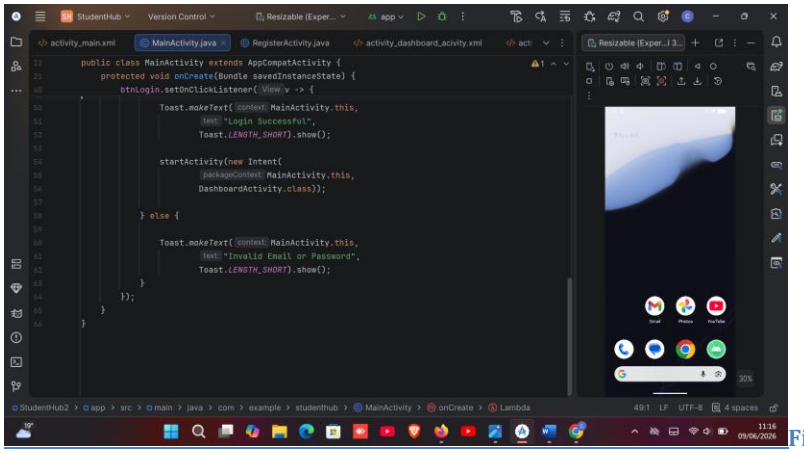
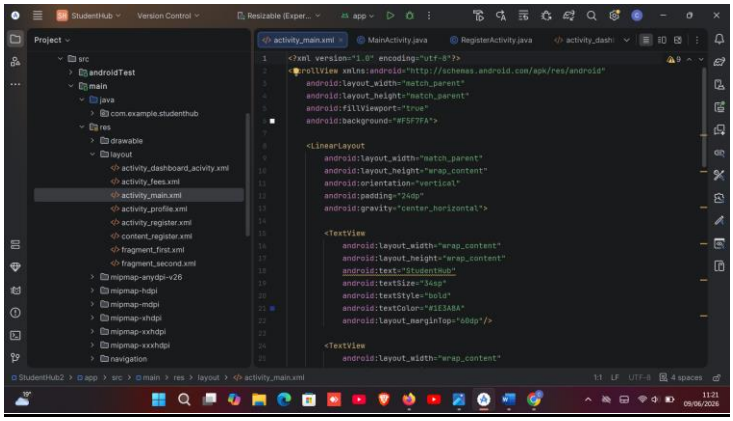
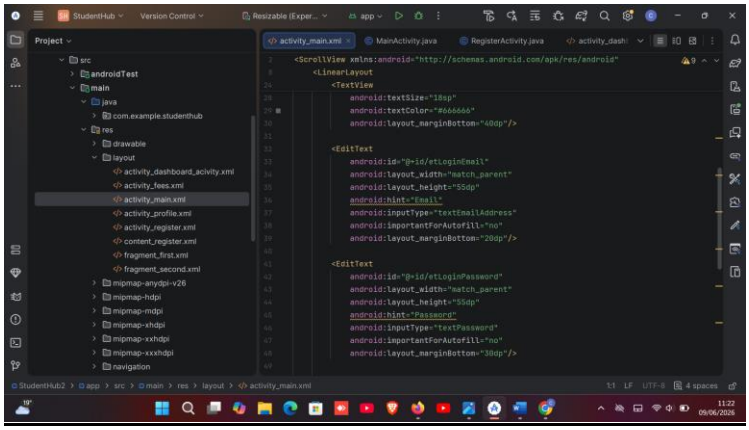


Figure 3.all are main activity.java codes

Xml layout code





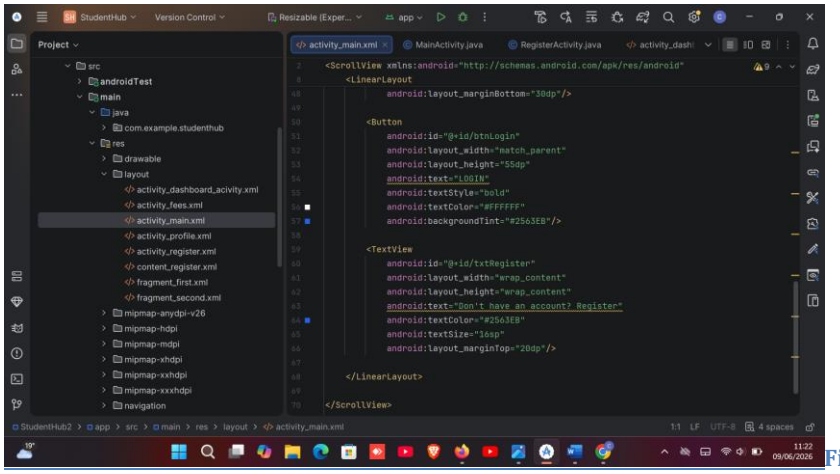


Figure 4.xml layout codes and design

Login Screen Design

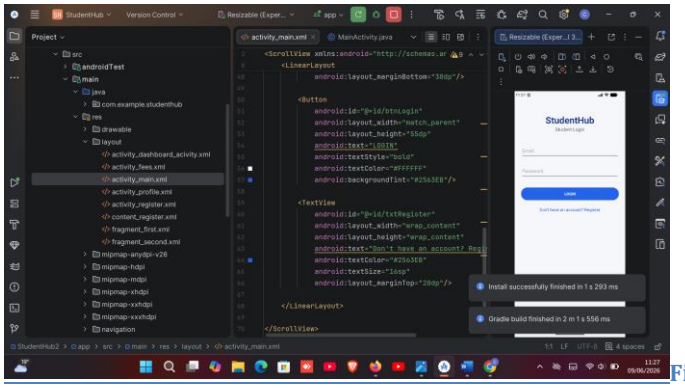


Figure 5login screen design

Student Registration Form

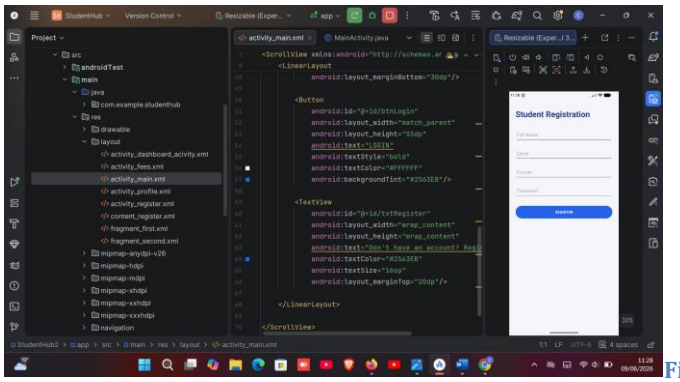
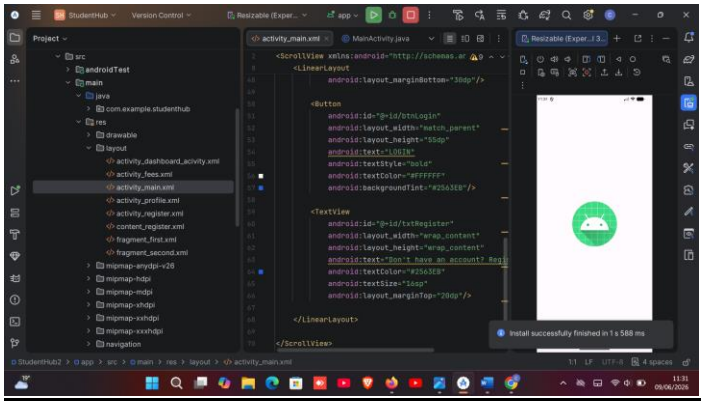


Figure 6.student registration form

Android Emulator Running Application



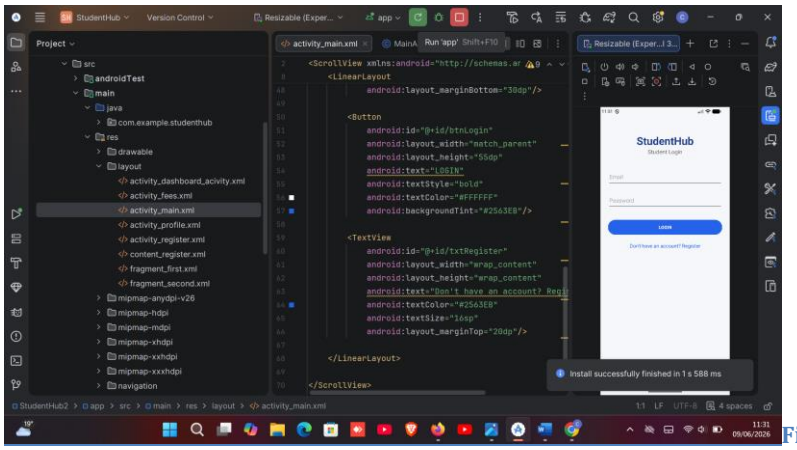


Figure 7 .app launching on emulator and running

Tools/Software Used

- Android Studio
- Java
- XML Layout Editor
- Android Emulator

Personal Reflection

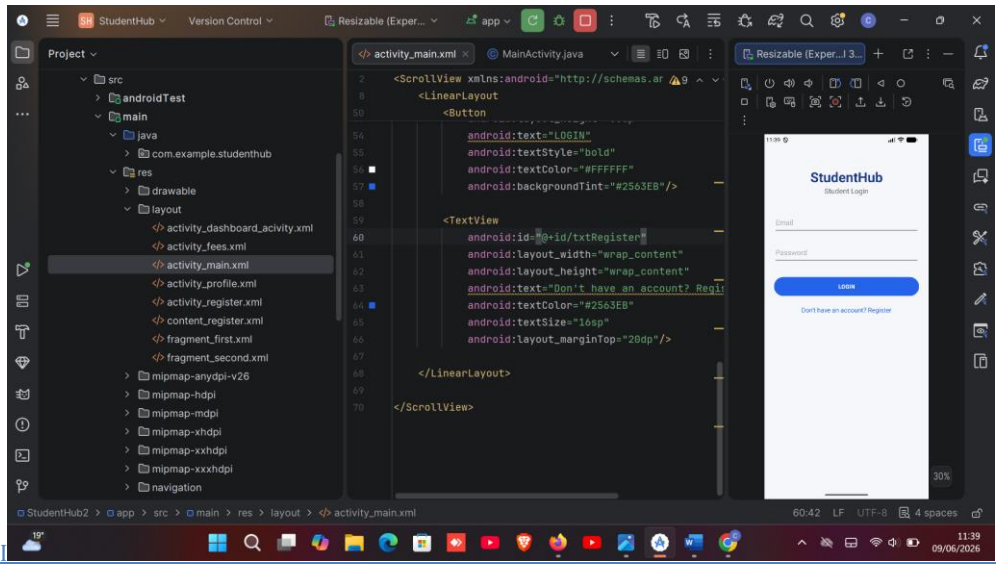
I improved my Java programming skills and learned how XML layouts work together with Java code. Designing the login and registration screens helped me understand Android user interface development.

Week 3: User Interface Design

Practical Activities Undertaken

1. Planned the Student Management App user interface.
2. Designed screen layouts using XML.
3. Created Login Screen.
4. Created Dashboard Screen.
5. Created Student Registration Form.
6. Implemented buttons and input fields.
7. Applied colors, spacing and alignment for a professional appearance.
8. Tested responsiveness of layouts on the Android emulator.
9. Improved navigation between screens.

Screenshots to Attach



Login Screen UI

Figure 8.login screen ui

Dashboard screen

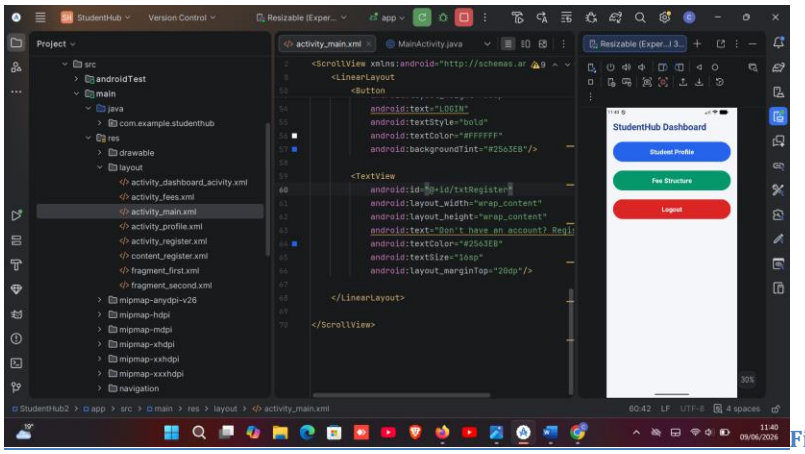


Figure 9.dashboard screen

Student registration screen

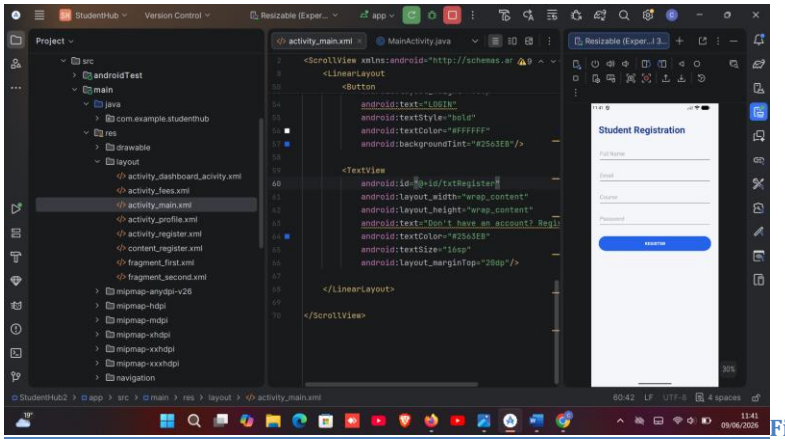
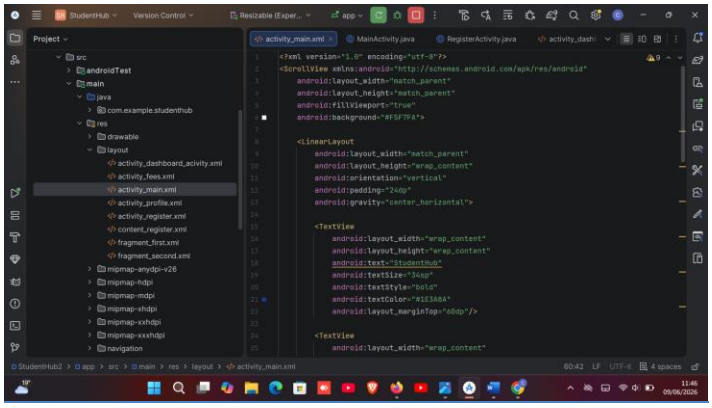
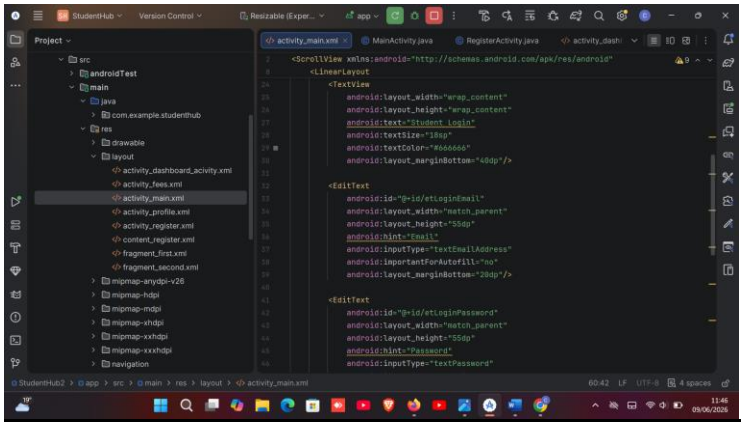


Figure 10.student registration screen

Xml code editor





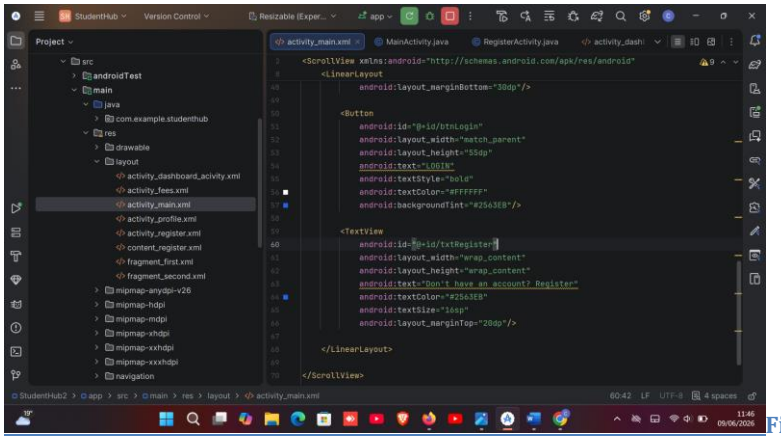


Figure 11.all xml editor codes

Navigation Between Screens

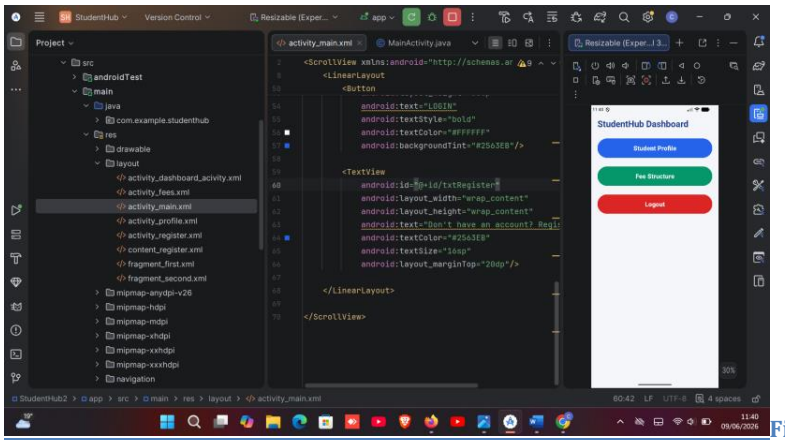


Figure 12.navigation btwn screens

Navigation to profile screens

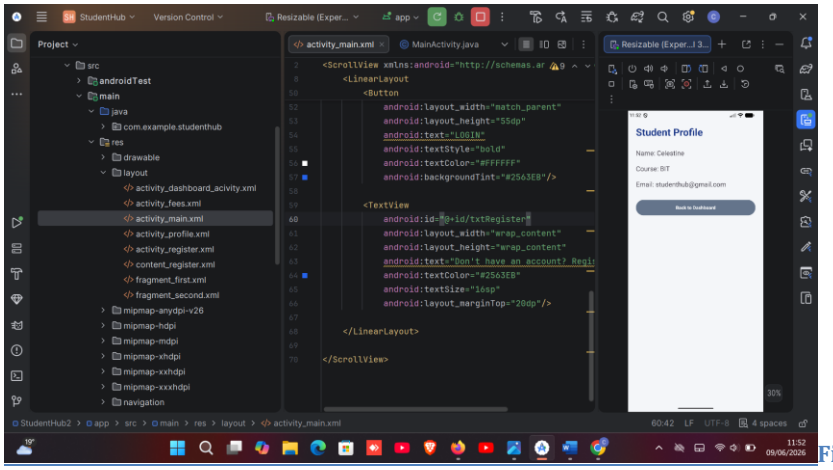


Figure 13.navigation to profile screens

Navigation to fee structure screen

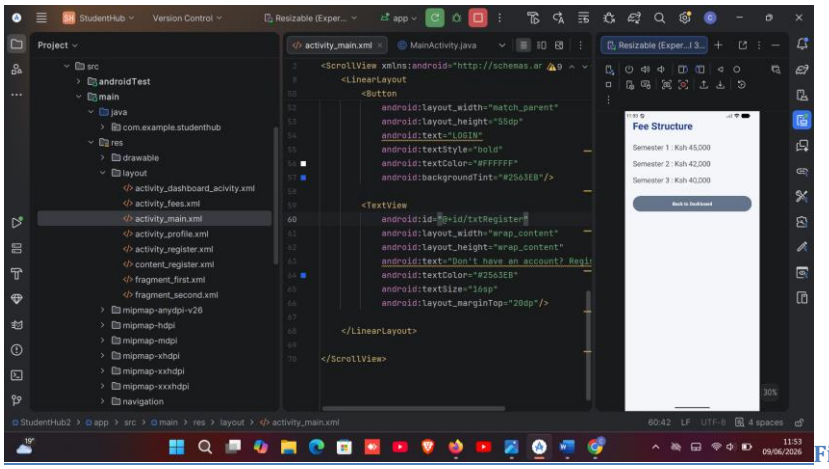
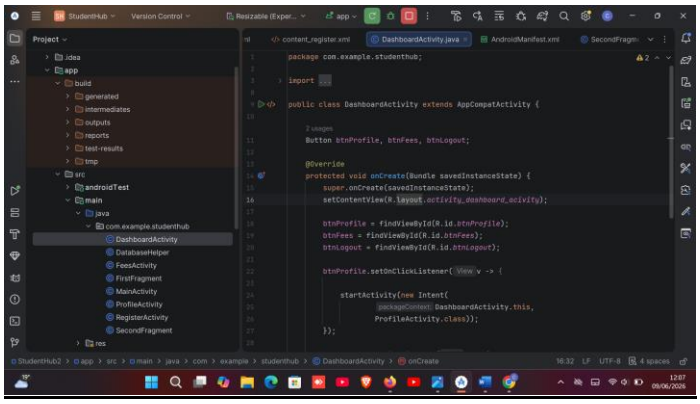


Figure 14.navigation to fee structure

Java code implementing button clicks



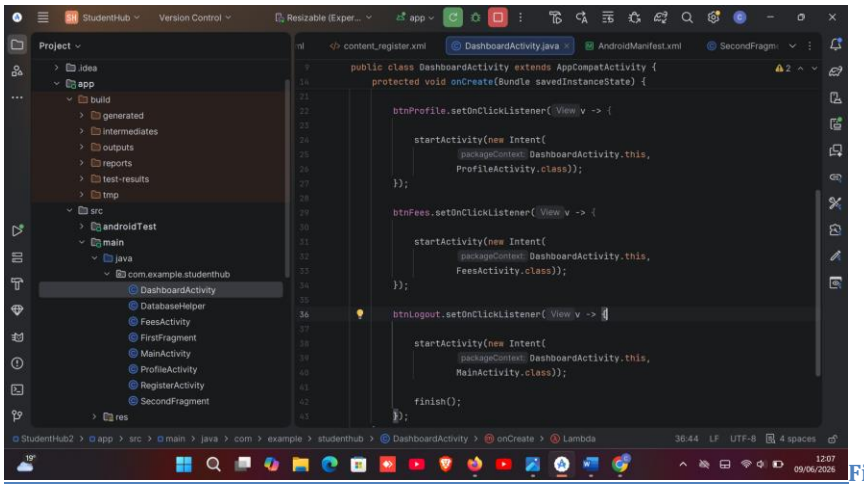


Figure 15.java code on button clicks

Tools/Software Used

- Android Studio
- Java
- XML
- Android Emulator

Personal Reflection

This week I focused on user interface design and learned how to create attractive Android layouts using XML. The practical work improved my ability to design user-friendly screens and navigation within the application.

Week 4: Event Handling and User Interaction

Practical Activities Undertaken

1. Enhanced the Student Hub dashboard by replacing simple navigation buttons with professional Material Card Views.
2. Implemented click event handling using On Click Listener for all dashboard modules.
3. Added navigation between Profile, Course Registration, GPA Estimator, Timetable, E-Learning Center, Announcements, Student Records, Settings, and Logout screens.
4. Created smooth activity transitions between screens.
5. Implemented form validation for student registration and data entry fields.
6. Added button interactions for GPA calculation and E-Learning resources.
7. Improved user experience through organized dashboard layout and modern interface design.
8. Tested all user interactions using the Android Emulator.
9. Fixed navigation and activity launch errors.
10. Verified correct operation of all dashboard modules.

Android Studio Project Structure

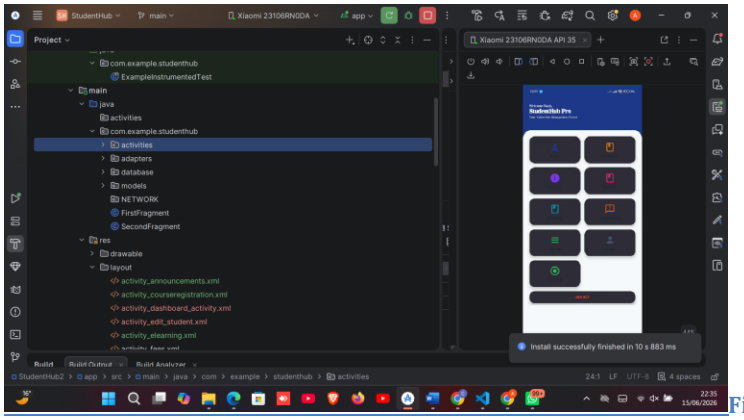
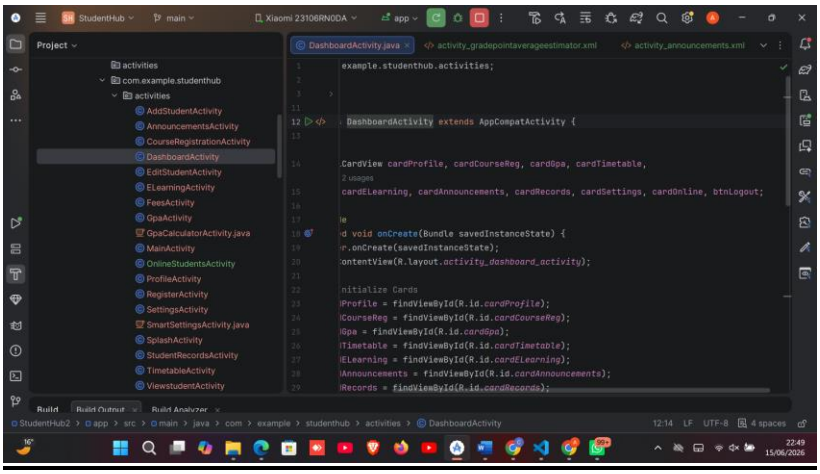
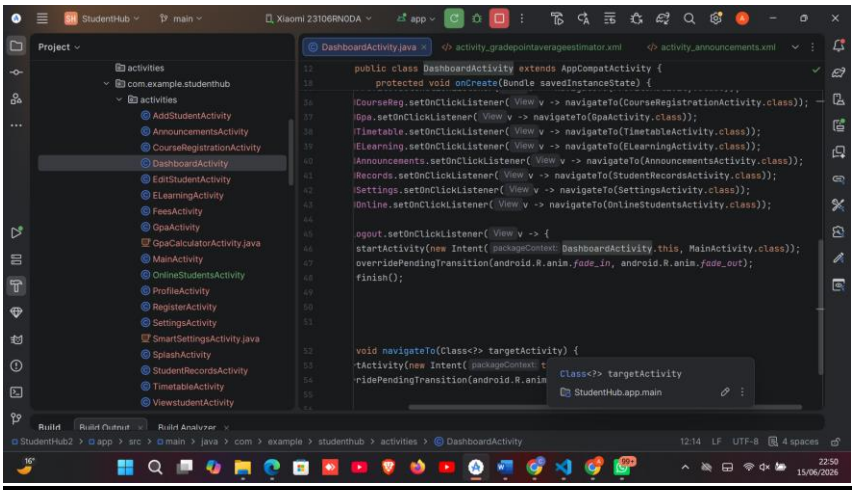


Figure 16ANDROID STUDIO STRUCTURE

DatabaseHelper.java


StudentHub
main
Xiaomi 23106RN0DA
app
Project
activities
com.example.studenthub
activities
AddStudentActivity
AnnouncementsActivity
CourseRegistrationActivity
DashboardActivity
EditStudentActivity
ELearningActivity
FeesActivity
GpaActivity
GpaCalculatorActivity.java
MainActivity
OnlineStudentsActivity
ProfileActivity
RegisterActivity
SettingsActivity
SmartSettingsActivity.java
SplashActivity
StudentRecordsActivity
TimetableActivity
ViewstudentActivity
DashboardActivity.java
activity_gradepointaverageestimator.xml
activity_announcements.xml
example.studenthub.activities;
DashboardActivity extends AppCompatActivity {
CardView cardProfile, cardCourseReg, cardGpa, cardTimetable,
2 usages
cardELearning, cardAnnouncements, cardRecords, cardSettings, cardOnline, btnLogout;
d void onCreate(Bundle savedInstanceState) {
.onCreate(savedInstanceState);
ontentView(R.layout.activity_dashboard_activity);
nitialize Cards
Profile = findViewById(R.id.cardProfile);
CourseReg = findViewById(R.id.cardCourseReg);
Gpa = findViewById(R.id.cardGpa);
Timetable = findViewById(R.id.cardTimetable);
ELearning = findViewById(R.id.cardELearning);
Announcements = findViewById(R.id.cardAnnouncements);
Records = findViewById(R.id.cardRecords);
Build
Build Output
Build Analyzer
StudentHub2 › app › src › main › java › com › example › studenthub › activities › DashboardActivity
12:14
LF
UTF-8
4 spaces
22:49
15/06/2026


StudentHub
main
Xiaomi 23106RN0DA
app
Project
activities
com.example.studenthub
activities
AddStudentActivity
AnnouncementsActivity
CourseRegistrationActivity
DashboardActivity
EditStudentActivity
ELearningActivity
FeesActivity
GpaActivity
GpaCalculatorActivity.java
MainActivity
OnlineStudentsActivity
ProfileActivity
RegisterActivity
SettingsActivity
SmartSettingsActivity.java
SplashActivity
StudentRecordsActivity
TimetableActivity
ViewstudentActivity
DashboardActivity.java
activity_gradepointaverageestimator.xml
activity_announcements.xml
public class DashboardActivity extends AppCompatActivity {
protected void onCreate(Bundle savedInstanceState) {
CourseReg.setOnClickListener(View v -> navigateTo(CourseRegistrationActivity.class));
Gpa.setOnClickListener(View v -> navigateTo(GpaActivity.class));
Timetable.setOnClickListener(View v -> navigateTo(TimetableActivity.class));
ELearning.setOnClickListener(View v -> navigateTo(ELearningActivity.class));
Announcements.setOnClickListener(View v -> navigateTo(AnnouncementsActivity.class));
Records.setOnClickListener(View v -> navigateTo(StudentRecordsActivity.class));
Settings.setOnClickListener(View v -> navigateTo(SettingsActivity.class));
Online.setOnClickListener(View v -> navigateTo(OnlineStudentsActivity.class));
ogout.setOnClickListener(View v -> {
startActivity(new Intent(packageContext: DashboardActivity.this, MainActivity.class));
overridePendingTransition(android.R.anim.fade_in, android.R.anim.fade_out);
finish();
void navigateTo(Class<?> targetActivity) {
tActivity(new Intent(packageContext: t
ridePendingTransition(android.R.anim
Class<?> targetActivity
StudentHub.app.main
Build
Build Output
Build Analyzer
StudentHub2 › app › src › main › java › com › example › studenthub › activities › DashboardActivity
12:14
LF
UTF-8
4 spaces
22:50
15/06/2026

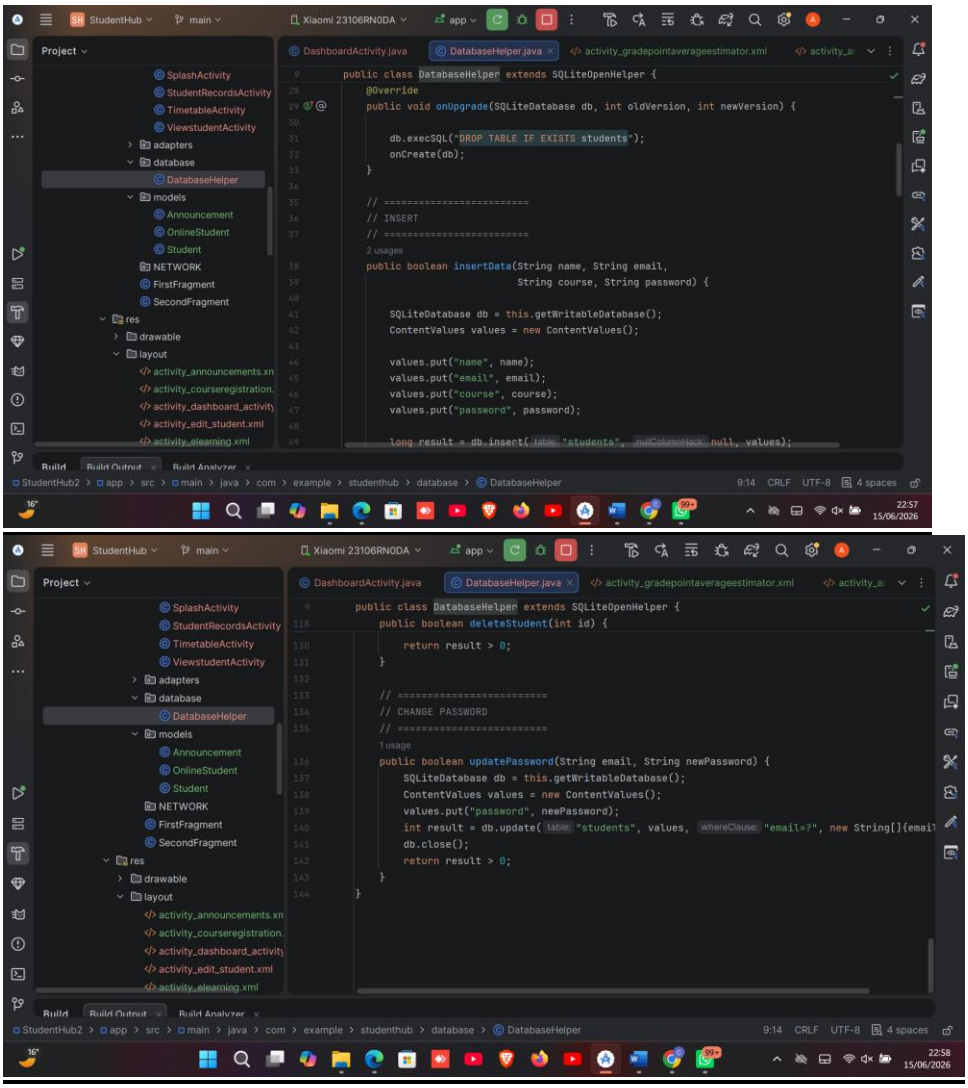
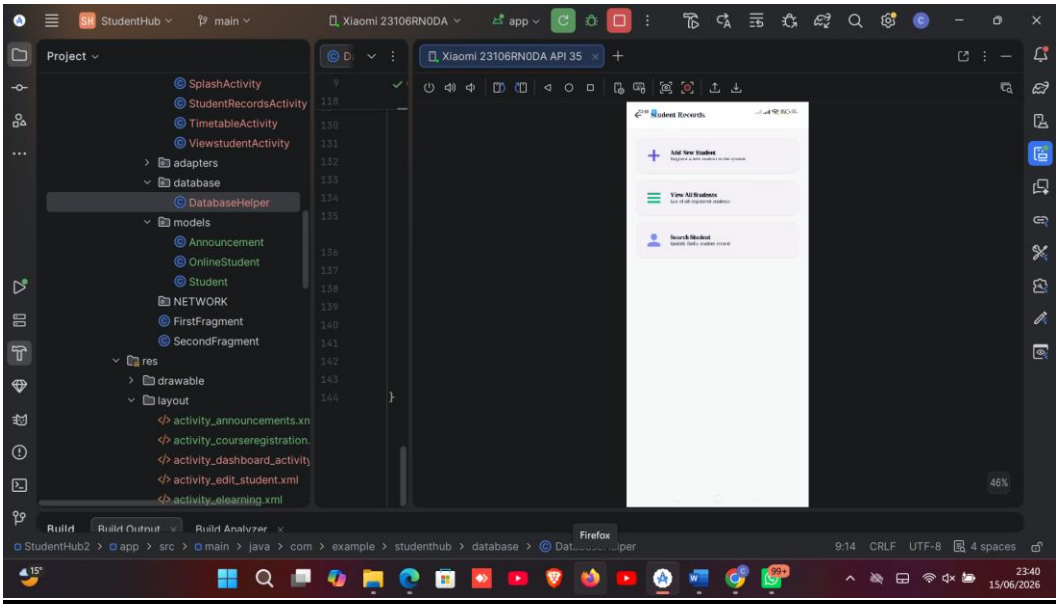


Figure 17DATABASE HELPER.JAVA CODES

Add Student Screen



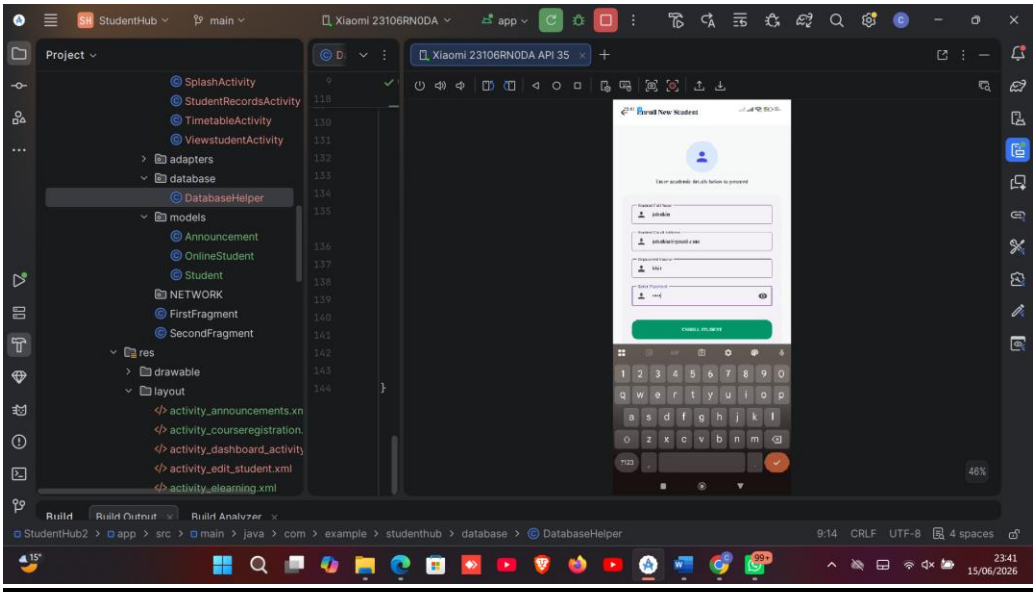


Figure 18ADD STUDENT

Student Saved Successfully

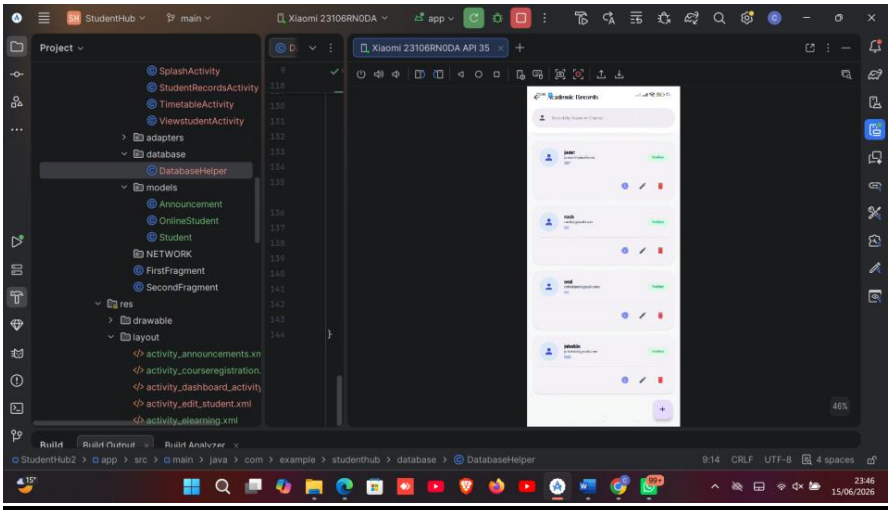


Figure 19STUDENT SAVED

View Students Screen

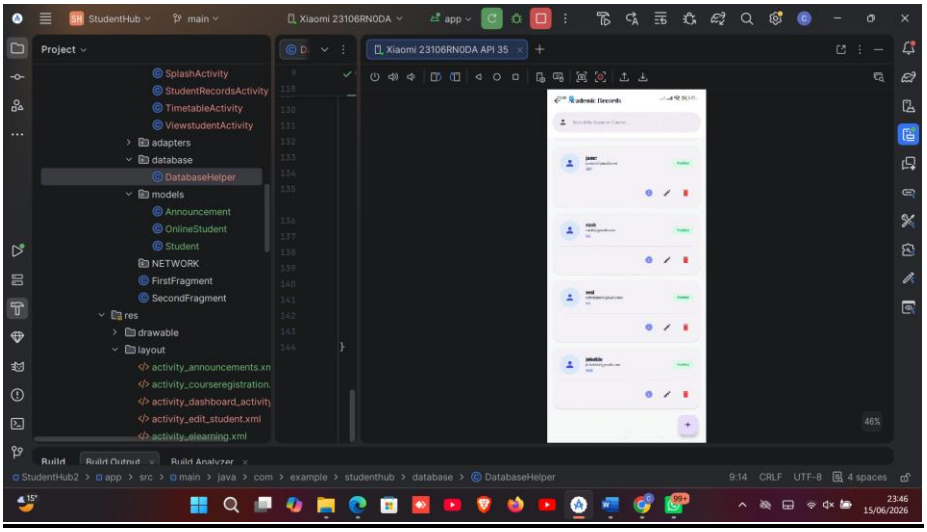
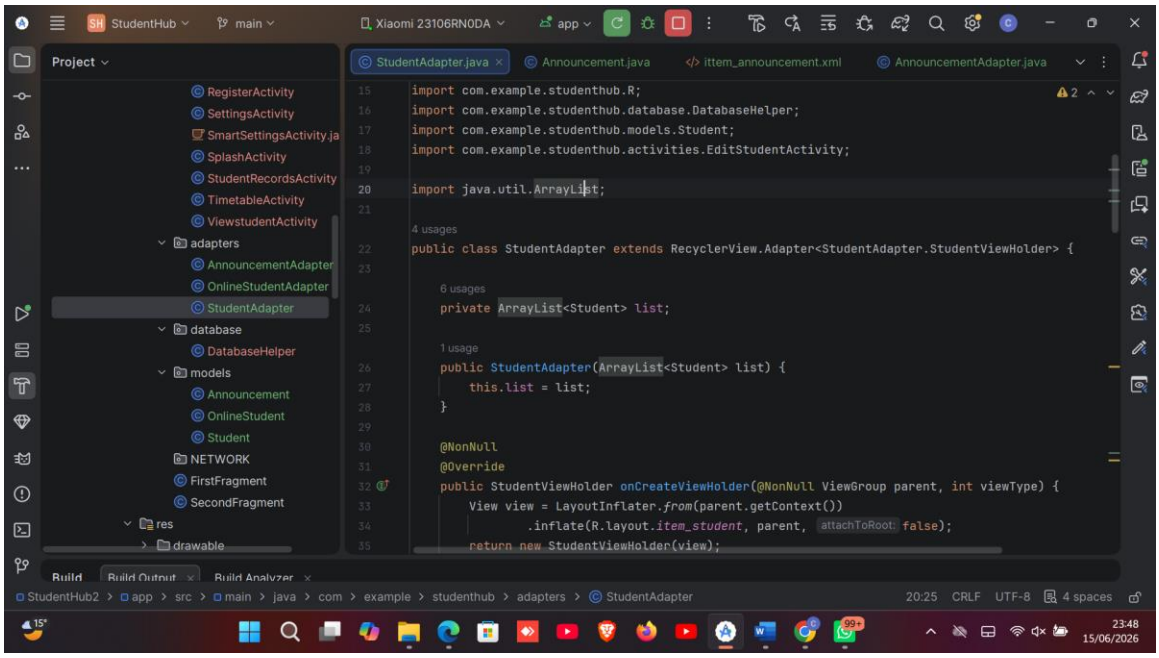
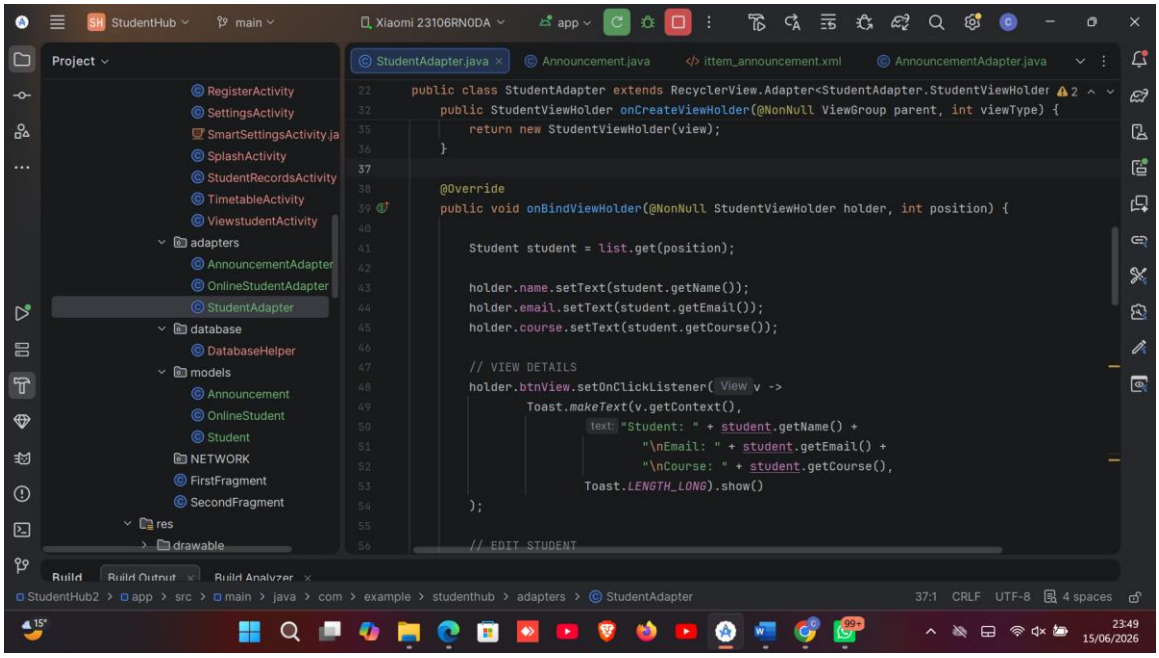
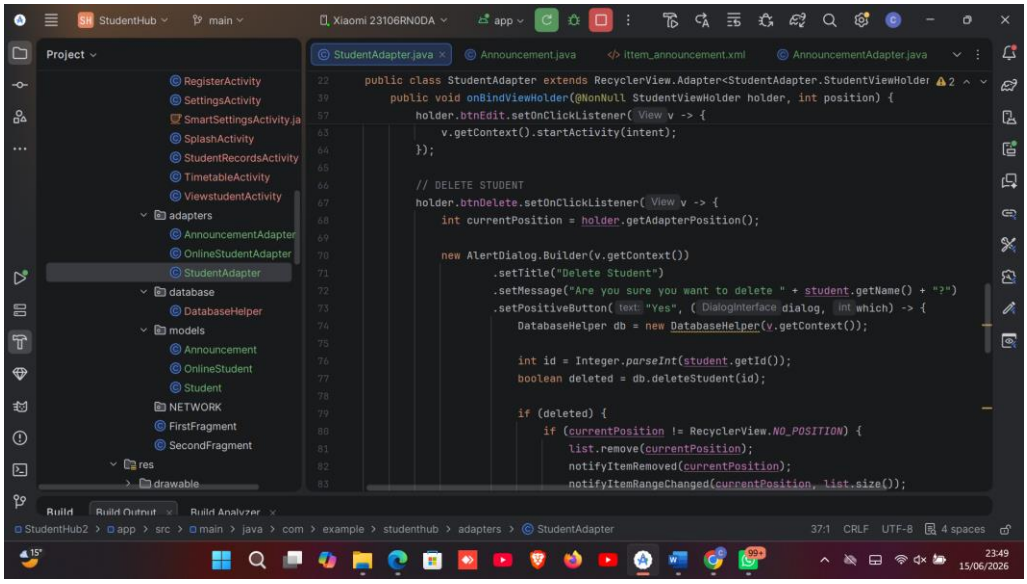


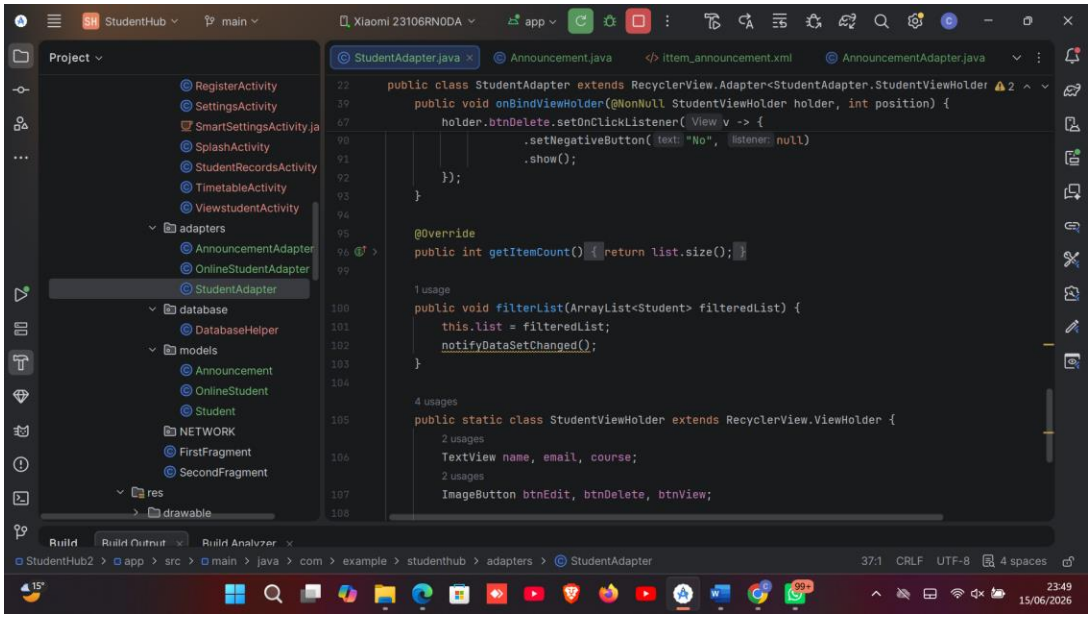
Figure 20Figure 21. RecyclerView displaying stored student records.

RecyclerView Adapter


StudentHub
main
Xiaomi 23106RN0DA
app
Project
StudentAdapter.java
Announcement.java
item_announcement.xml
AnnouncementAdapter.java
RegisterActivity
SettingsActivity
SmartSettingsActivity.ja
SplashActivity
StudentRecordsActivity
TimetableActivity
ViewstudentActivity
adapters
AnnouncementAdapter
OnlineStudentAdapter
StudentAdapter
database
DatabaseHelper
models
Announcement
OnlineStudent
Student
NETWORK
FirstFragment
SecondFragment
res
drawable
import com.example.studenthub.R;
import com.example.studenthub.database.DatabaseHelper;
import com.example.studenthub.models.Student;
import com.example.studenthub.activities.EditStudentActivity;
import java.util.ArrayList;
4 usages
public class StudentAdapter extends RecyclerView.Adapter<StudentAdapter.StudentViewHolder> {
6 usages
private ArrayList<Student> list;
1 usage
public StudentAdapter(ArrayList<Student> list) {
this.list = list;
}
@NonNull
@Override
public StudentViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
View view = LayoutInflater.from(parent.getContext())
.inflate(R.layout.item_student, parent, attachToRoot: false);
return new StudentViewHolder(view);
Build
Build Output
Build Analyzer
StudentHub2 > app > src > main > java > com > example > studenthub > adapters > StudentAdapter
20:25
CRLF
UTF-8
4 spaces
23:48
15/06/2026


StudentHub
main
Xiaomi 23106RN0DA
app
Project
StudentAdapter.java
Announcement.java
item_announcement.xml
AnnouncementAdapter.java
RegisterActivity
SettingsActivity
SmartSettingsActivity.ja
SplashActivity
StudentRecordsActivity
TimetableActivity
ViewstudentActivity
adapters
AnnouncementAdapter
OnlineStudentAdapter
StudentAdapter
database
DatabaseHelper
models
Announcement
OnlineStudent
Student
NETWORK
FirstFragment
SecondFragment
res
drawable
public class StudentAdapter extends RecyclerView.Adapter<StudentAdapter.StudentViewHolder
public StudentViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
return new StudentViewHolder(view);
}
@Override
public void onBindViewHolder(@NonNull StudentViewHolder holder, int position) {
Student student = list.get(position);
holder.name.setText(student.getName());
holder.email.setText(student.getEmail());
holder.course.setText(student.getCourse());
// VIEW DETAILS
holder.btnView.setOnClickListener(View v ->
Toast.makeText(v.getContext(),
text: "Student: " + student.getName() +
"\nEmail: " + student.getEmail() +
"\nCourse: " + student.getCourse(),
Toast.LENGTH_LONG).show()
);
// EDIT STUDENT
Build
Build Output
Build Analyzer
StudentHub2 > app > src > main > java > com > example > studenthub > adapters > StudentAdapter
37:1
CRLF
UTF-8
4 spaces
23:49
15/06/2026


StudentHub
main
Xiaomi 23106RN0DA
app
Project
StudentAdapter.java
Announcement.java
item_announcement.xml
AnnouncementAdapter.java
RegisterActivity
SettingsActivity
SmartSettingsActivity.ja
SplashActivity
StudentRecordsActivity
TimetableActivity
ViewstudentActivity
adapters
AnnouncementAdapter
OnlineStudentAdapter
StudentAdapter
database
DatabaseHelper
models
Announcement
OnlineStudent
Student
NETWORK
FirstFragment
SecondFragment
res
drawable
public class StudentAdapter extends RecyclerView.Adapter<StudentAdapter.StudentViewHolder
public void onBindViewHolder(@NonNull StudentViewHolder holder, int position) {
holder.btnEdit.setOnClickListener(View v -> {
v.getContext().startActivity(intent);
});
// DELETE STUDENT
holder.btnDelete.setOnClickListener(View v -> {
int currentPosition = holder.getAdapterPosition();
new AlertDialog.Builder(v.getContext())
.setTitle("Delete Student")
.setMessage("Are you sure you want to delete " + student.getName() + "?")
.setPositiveButton(text: "Yes", (DialogInterface dialog, int which) -> {
DatabaseHelper db = new DatabaseHelper(v.getContext());
int id = Integer.parseInt(student.getId());
boolean deleted = db.deleteStudent(id);
if (deleted) {
if (currentPosition != RecyclerView.NO_POSITION) {
list.remove(currentPosition);
notifyItemRemoved(currentPosition);
notifyItemRangeChanged(currentPosition, list.size());
Build
Build Output
Build Analyzer
StudentHub2 › app › src › main › java › com › example › studenthub › adapters › StudentAdapter
37:1 CRLF UTF-8 4 spaces
23:49
15/06/2026


StudentHub
main
Xiaomi 23106RN0DA
app
Project
StudentAdapter.java
Announcement.java
item_announcement.xml
AnnouncementAdapter.java
RegisterActivity
SettingsActivity
SmartSettingsActivity.ja
SplashActivity
StudentRecordsActivity
TimetableActivity
ViewstudentActivity
adapters
AnnouncementAdapter
OnlineStudentAdapter
StudentAdapter
database
DatabaseHelper
models
Announcement
OnlineStudent
Student
NETWORK
FirstFragment
SecondFragment
res
drawable
public class StudentAdapter extends RecyclerView.Adapter<StudentAdapter.StudentViewHolder
public void onBindViewHolder(@NonNull StudentViewHolder holder, int position) {
holder.btnDelete.setOnClickListener(View v -> {
.setNegativeButton(text: "No", listener: null)
.show();
});
}
@Override
public int getItemCount() { return list.size(); }
1 usage
public void filterList(ArrayList<Student> filteredList) {
this.list = filteredList;
notifyDataSetChanged();
}
4 usages
public static class StudentViewHolder extends RecyclerView.ViewHolder {
2 usages
TextView name, email, course;
2 usages
ImageButton btnEdit, btnDelete, btnView;
Build
Build Output
Build Analyzer
StudentHub2 › app › src › main › java › com › example › studenthub › adapters › StudentAdapter
37:1 CRLF UTF-8 4 spaces
23:49
15/06/2026

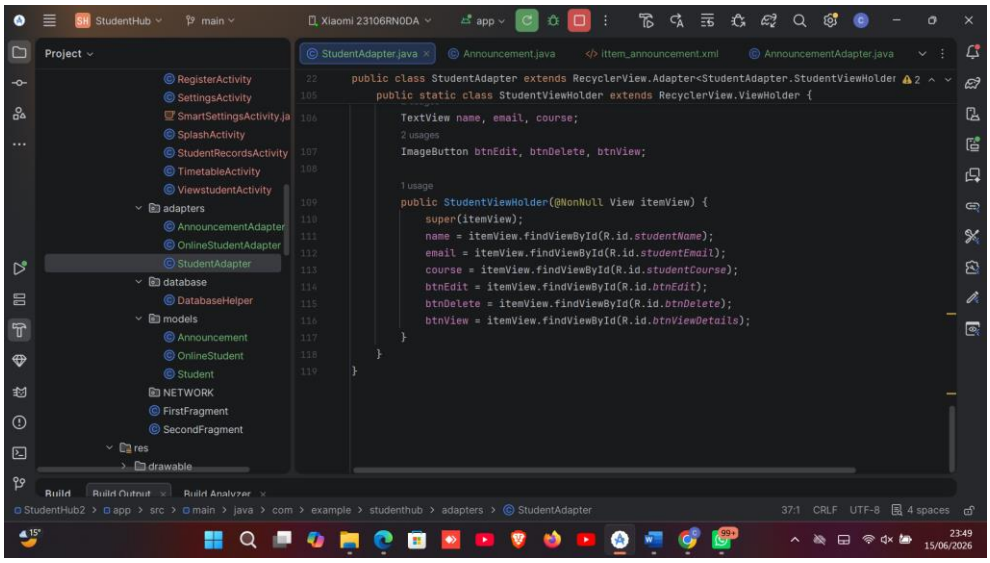


Figure 21Figure 22. RecyclerView adapter implementation.

Update Student

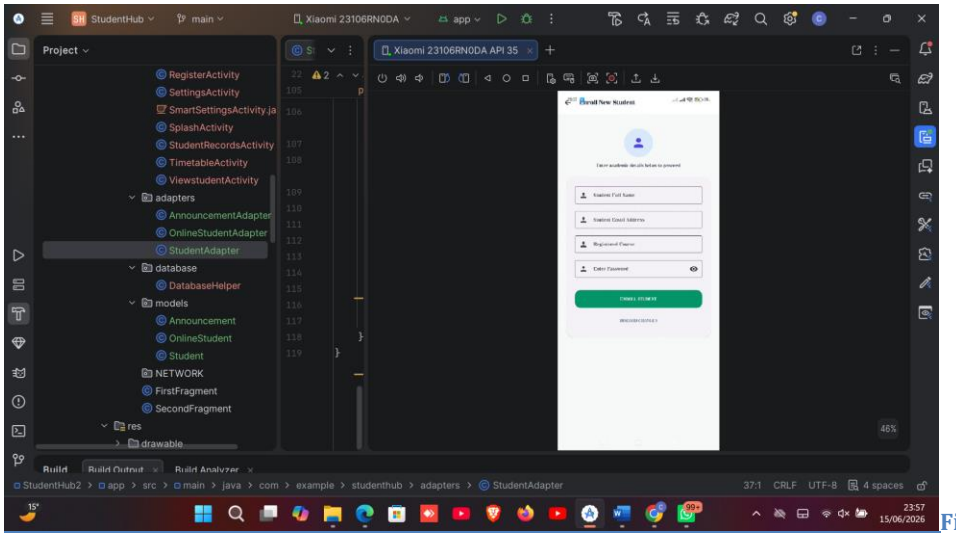
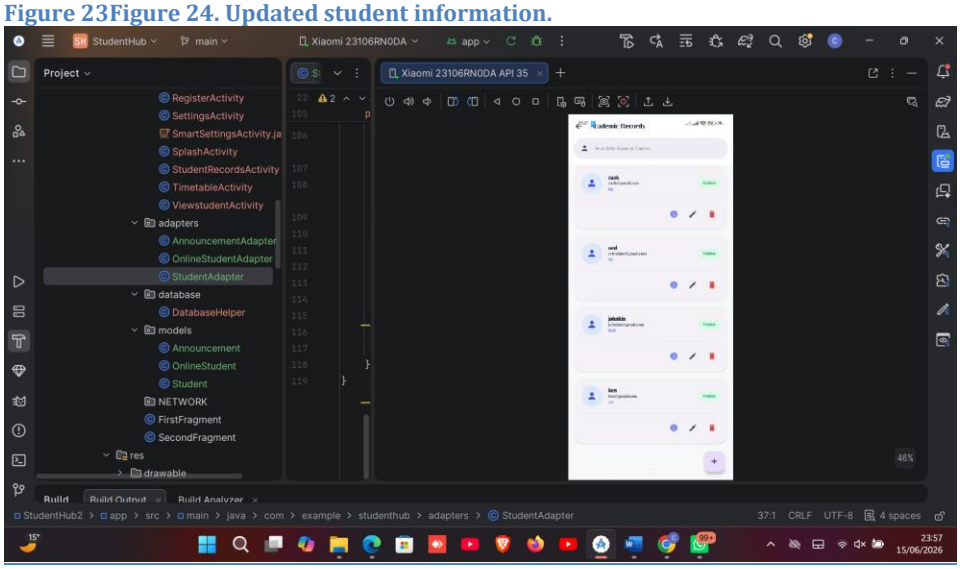


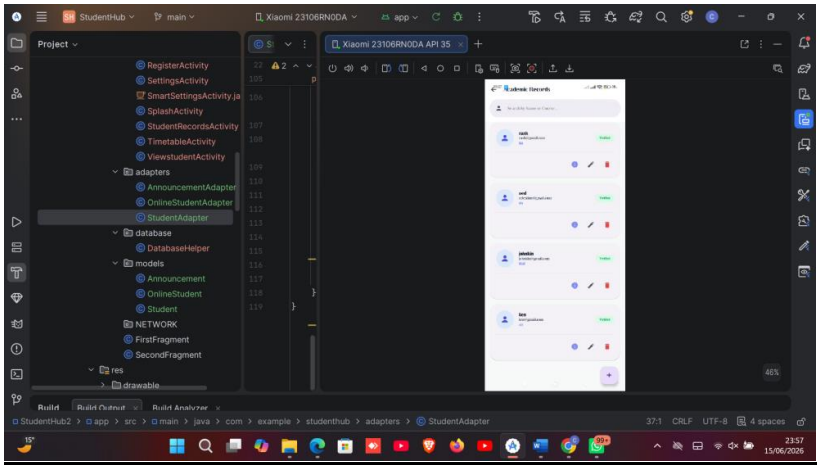
Figure 22Figure 23. Student record update screen.

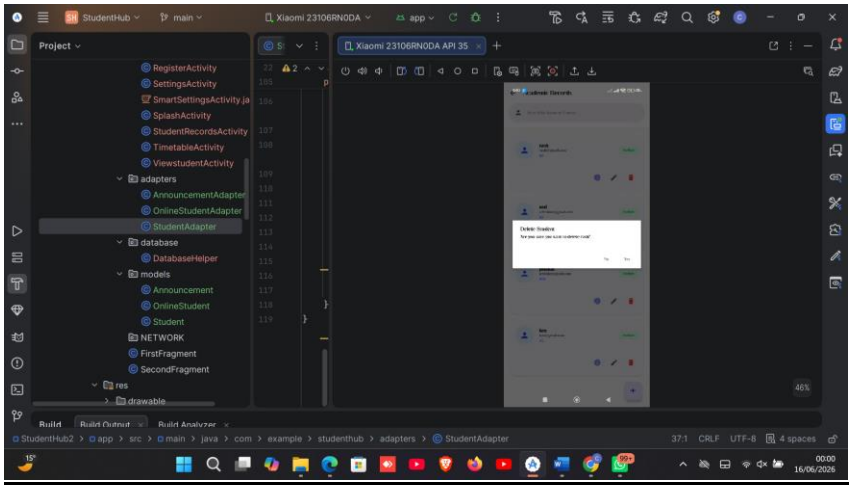
Updated record

Figure 23Figure 24. Updated student information.



Delete Function





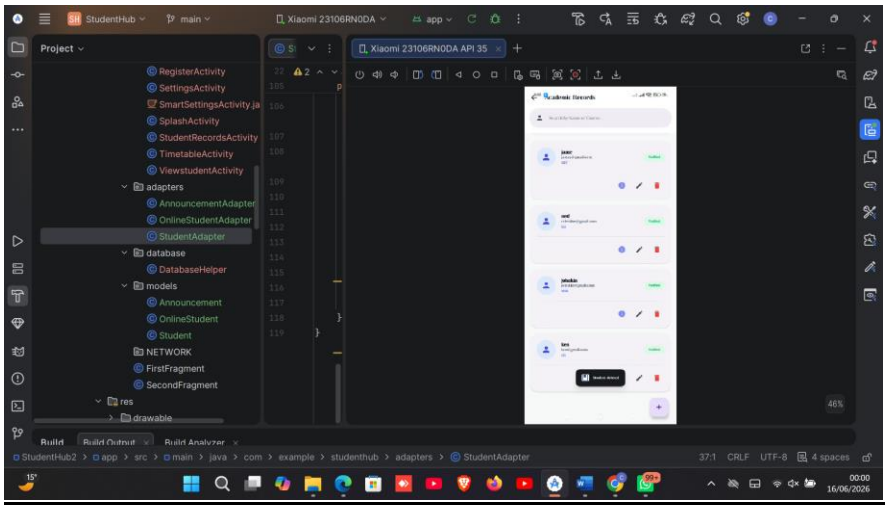
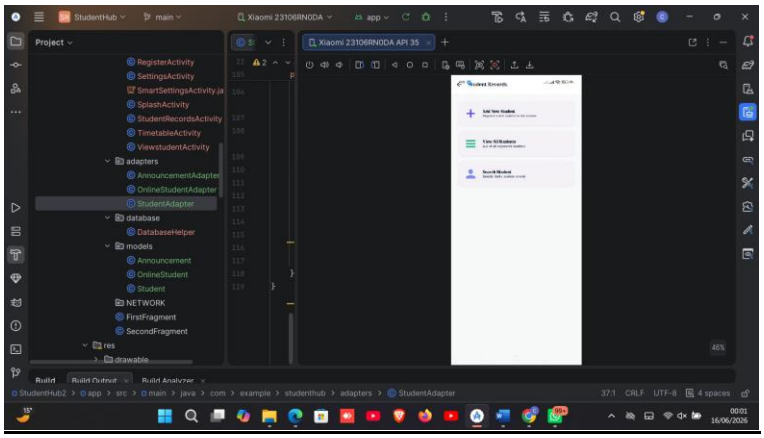
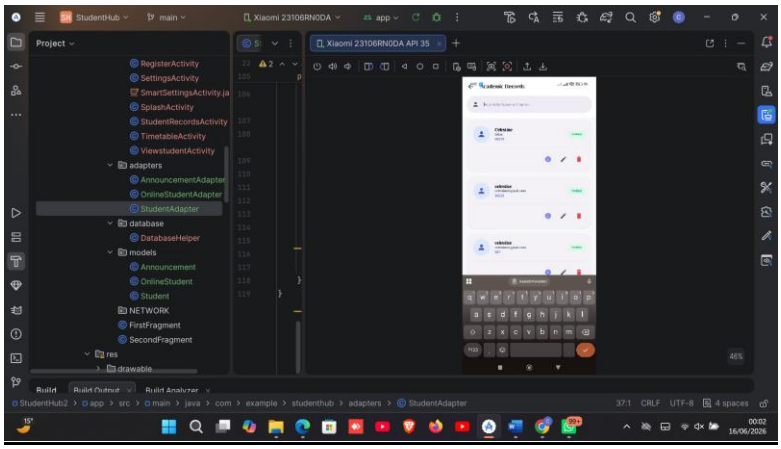


Figure 24Figure 25. Delete student functionality.

Search Function





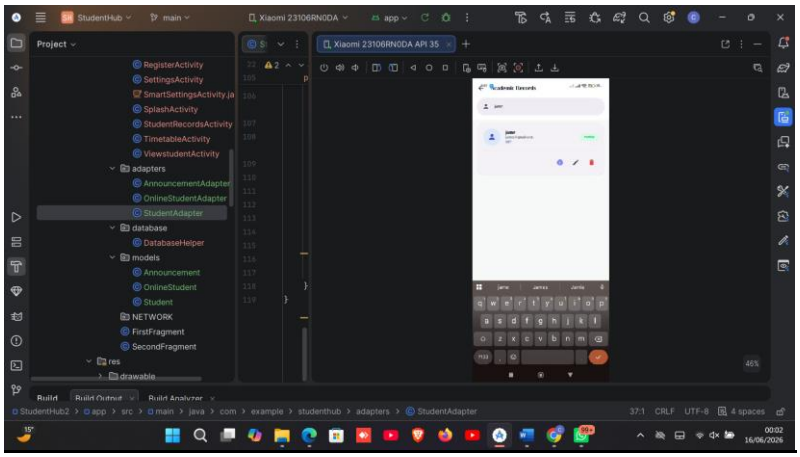
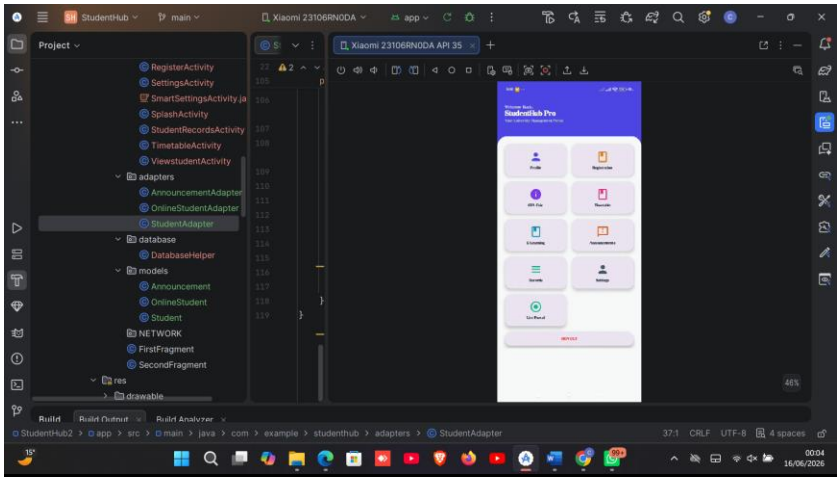


Figure 25Figure 26. Student search functionality.

Dashboard With New Modules



Tools/Software Used

- Android Studio
- Java Programming Language
- XML Layout Editor
- SQLite Database
- RecyclerView
- Android Emulator
- Material Design Components

PRACTICAL ACTIVITIES

Designed and implemented SQLite database storage for Student Hub. Created a students table and implemented CRUD operations including adding, viewing, updating, deleting, and searching student records. Developed a RecyclerView-based records screen to display stored information efficiently. Expanded the Student Hub dashboard by adding Student Records, GPA Estimator, Timetable, E-Learning, and Settings modules. Tested all database operations and verified successful storage and retrieval of student information.

Week 5: Data Management

Practical Activities Undertaken

1. Designed and implemented an SQLite database for the Student Hub application.
2. Created a DatabaseHelper class to manage database operations.
3. Developed a students table for storing student records.
4. Implemented student registration and data storage functionality.

5. Implemented CRUD operations (Create, Read, Update, Delete).
6. Developed a RecyclerView to display student records.
7. Added student search functionality for easy record retrieval.
8. Integrated the Student Records module with the dashboard.
9. Tested database storage and retrieval processes.
10. Verified successful updating and deletion of student records.

Announcements Module

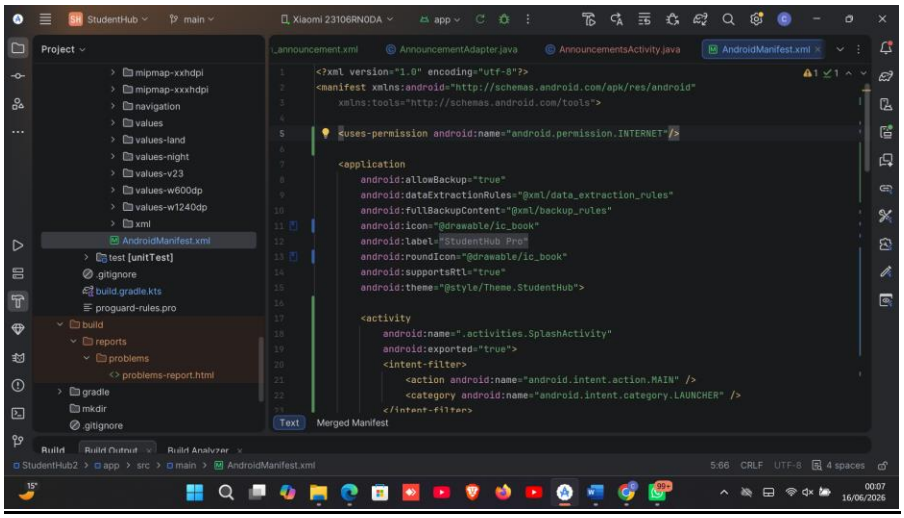
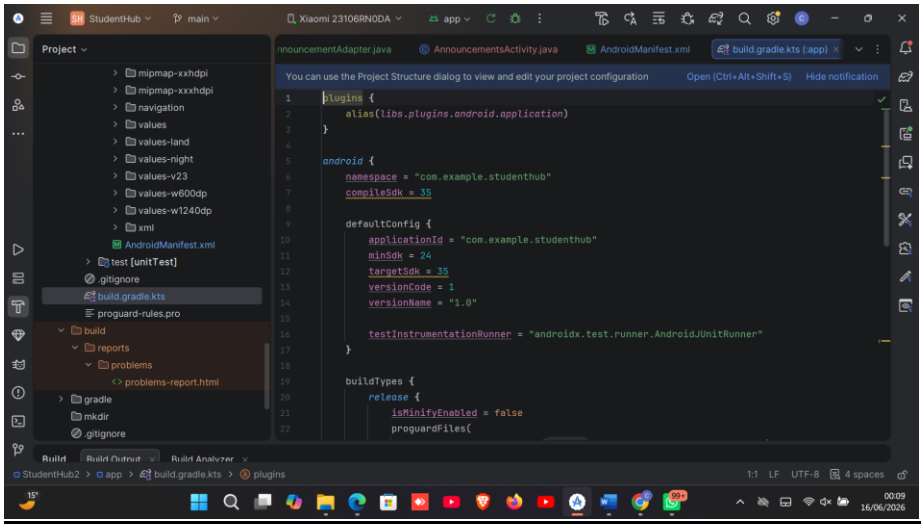


Figure 26ANNOUNCEMENT MODULE

build.gradle



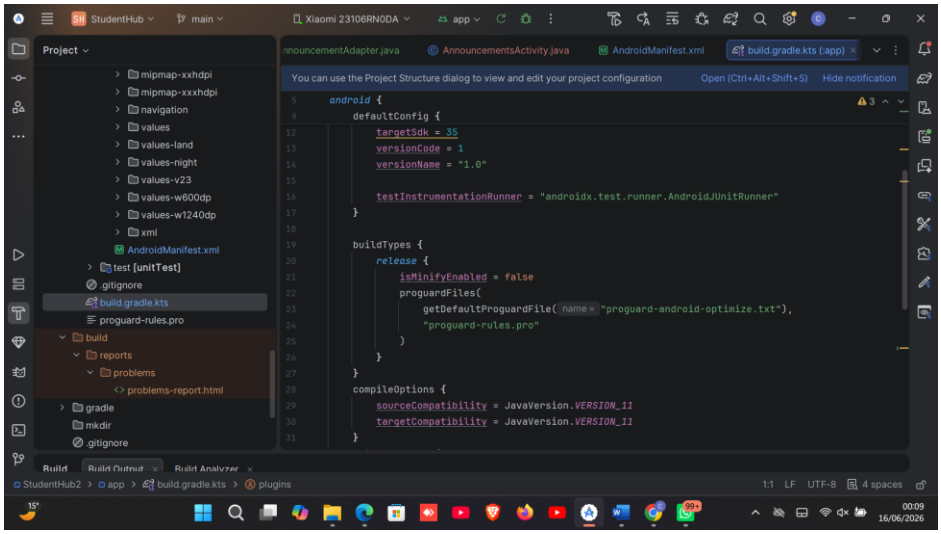
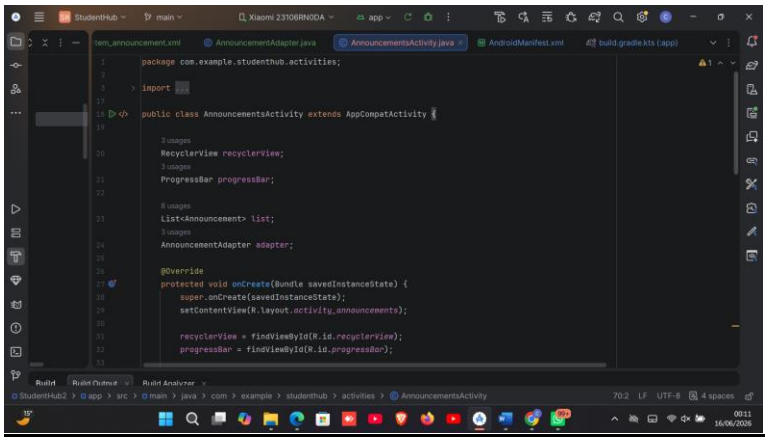


Figure 27BUILD GRADLE KTS

Announcement.java code



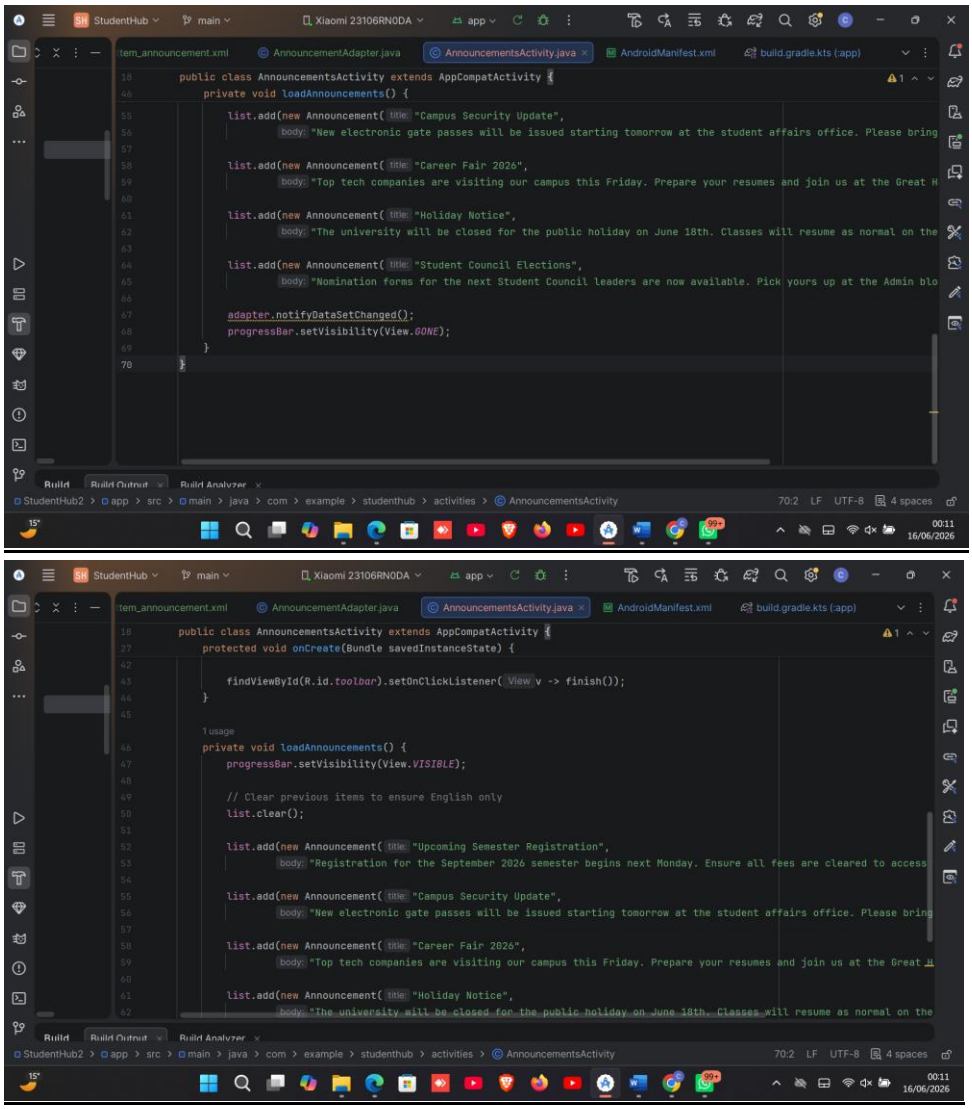
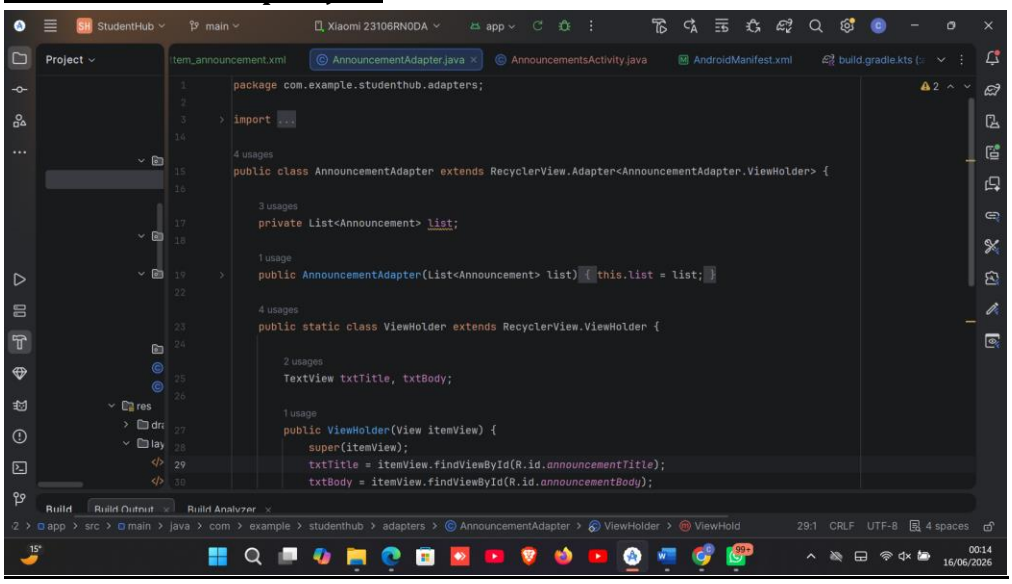


Figure 28ANNOUNCEMENT JAVA CODE

AnnouncementAdapter.java



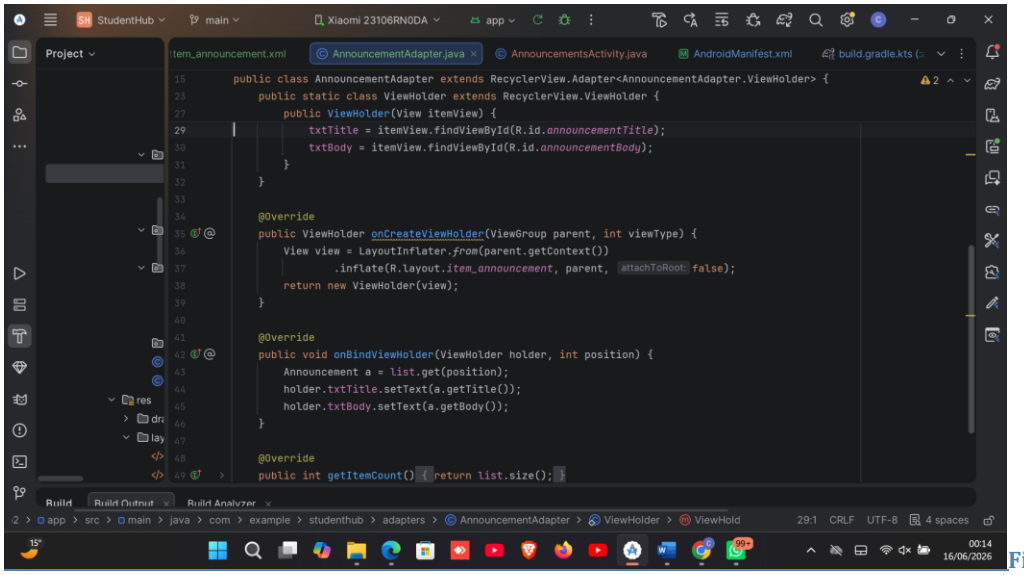


Figure 29ANNOUNCEMENT ADAPTER.JAVA

Show loaded announcements.

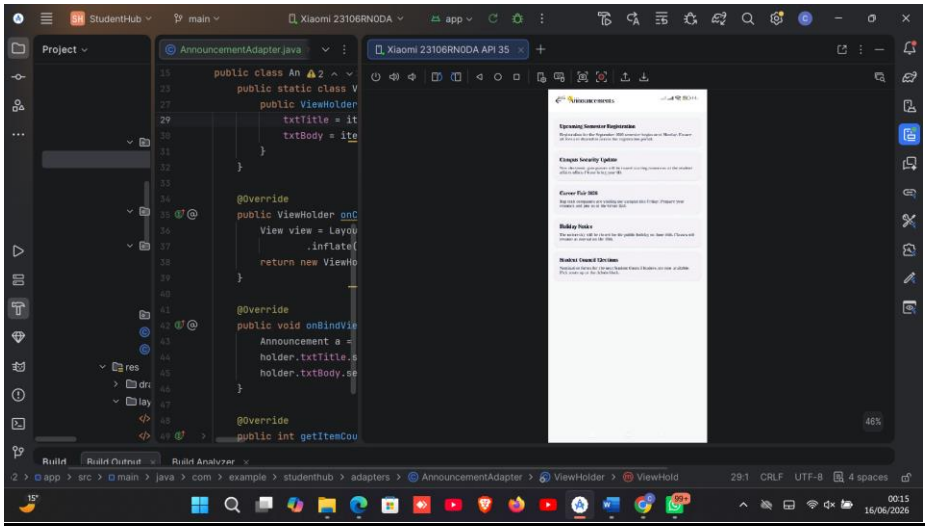


Figure 30ANNOUNCEMENT SCREEN

Live portal Figure JSON data parsing.

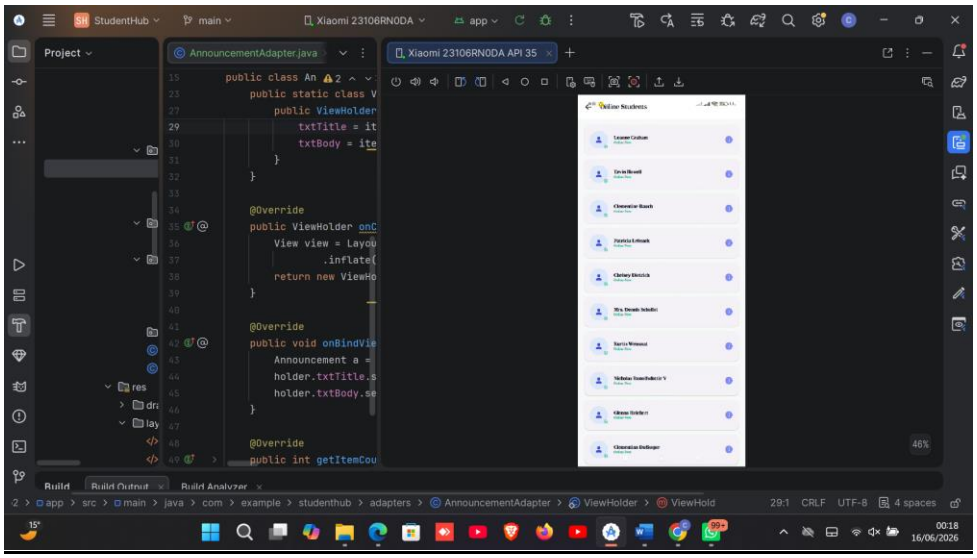


Figure 31LIVE PORTAL SCREEN

Tools/Software Used

- Android Studio
- Java Programming Language
- XML Layout Editor
- Volley Networking Library
- REST API (JSONPlaceholder API)

- JSON Data Format
- RecyclerView
- Android Emulator
- Internet Connectivity Tools (HTTP Requests)

PERSONAL REFLECTION

I gained practical experience in database management using SQLite. Implementing CRUD operations and RecyclerView improved my understanding of data storage, retrieval, and presentation in Android applications. This enhanced the functionality and reliability of the Student Hub project.

Week 6: Networking and APIs

Practical Activities Undertaken

1. Studied networking concepts and APIs.
2. Configured internet permissions in the application.
3. Added the Volley networking library.
4. Created models and RecyclerView adapters for online data.
5. Implemented HTTP requests and consumed a REST API.
6. Parsed JSON responses and displayed data in RecyclerView.
7. Implemented network error handling.
8. Integrated API features into the Announcements and Live Portal modules.
9. Tested API connectivity and data retrieval.

Show project structure:

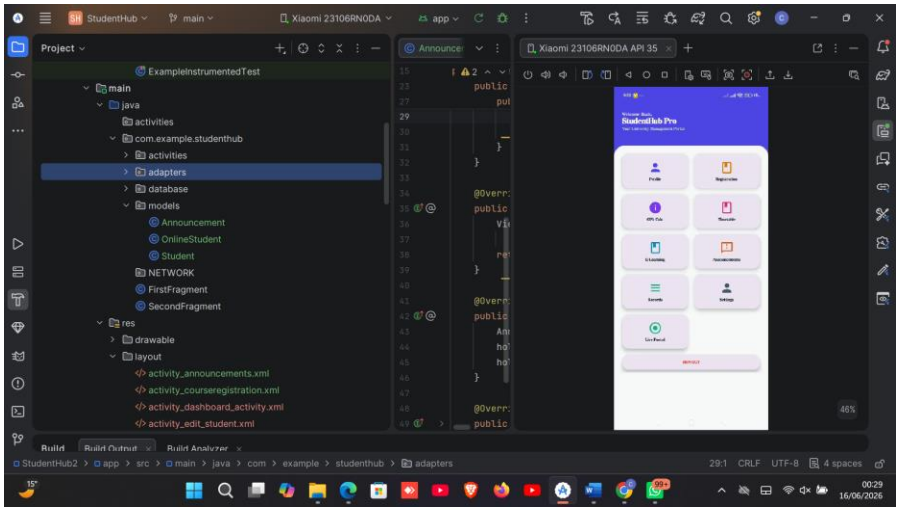


Figure 32PROJECT STRUCTURE

API MODEL

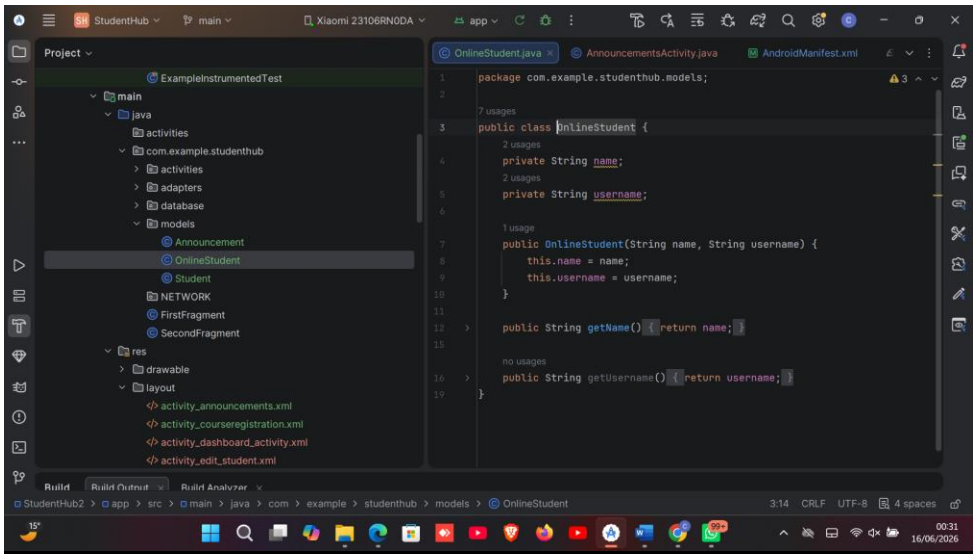
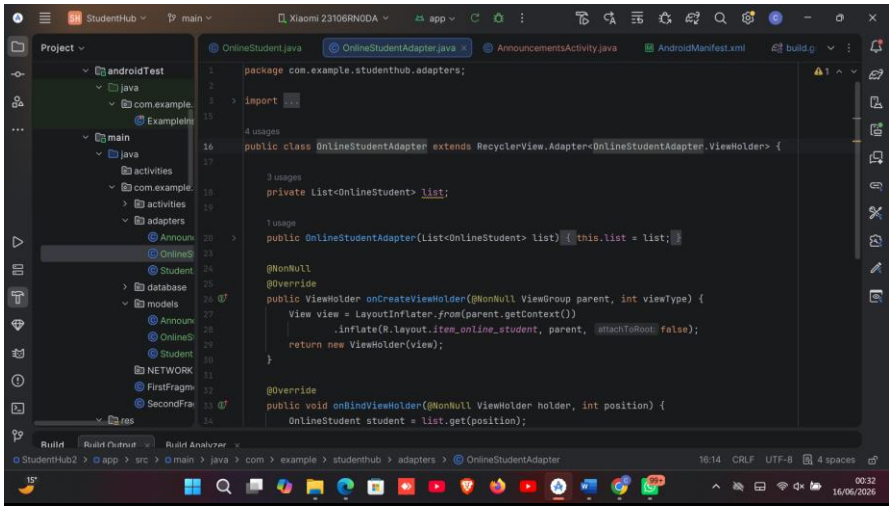


Figure 33API MODEL

API ADAPTER



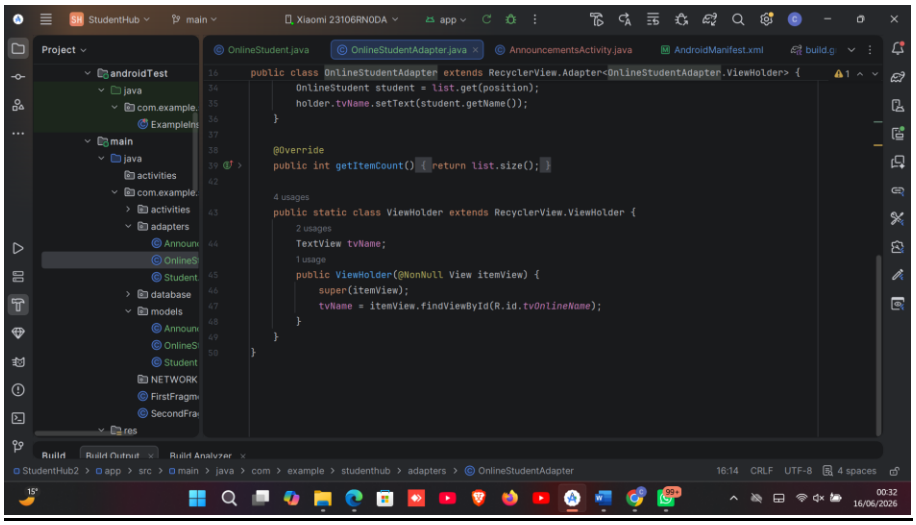
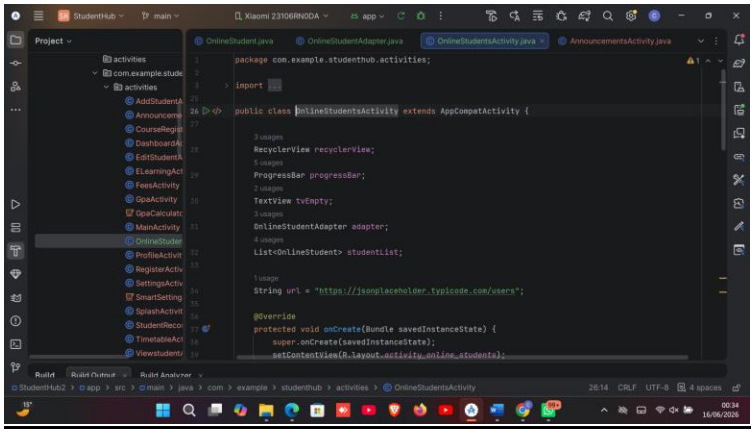
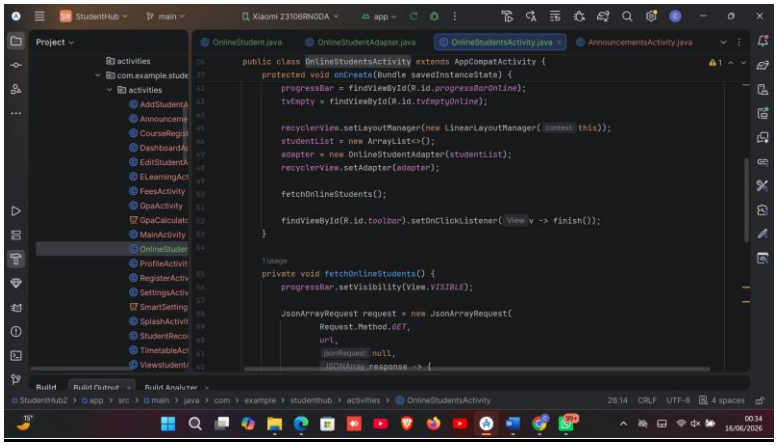


Figure 34API ADAPTER

API REQUEST CODE





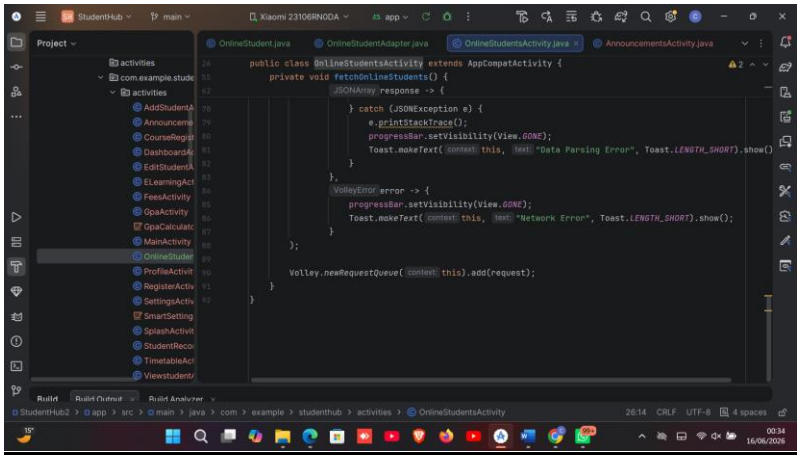


Figure 35Figure 41. HTTP GET request implementation using Volley.

RUN API SCREEN

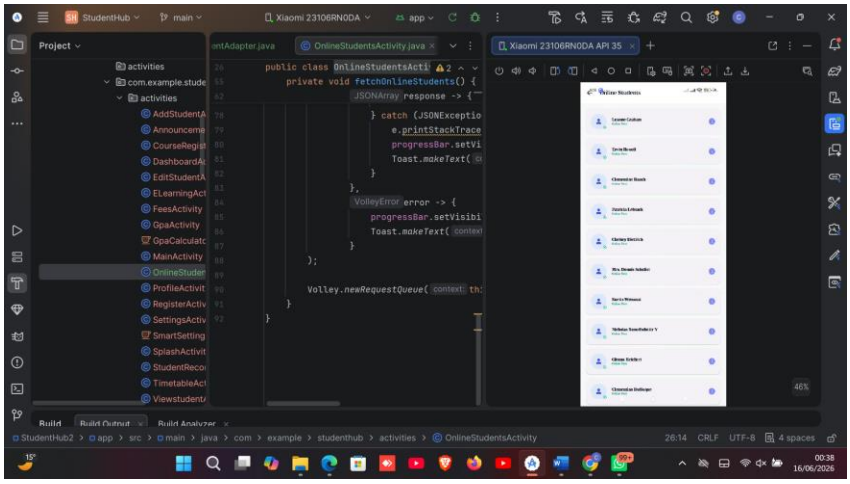


Figure 36Figure 43. API records successfully loaded from the internet.

SHOW RECYCLERVIEW

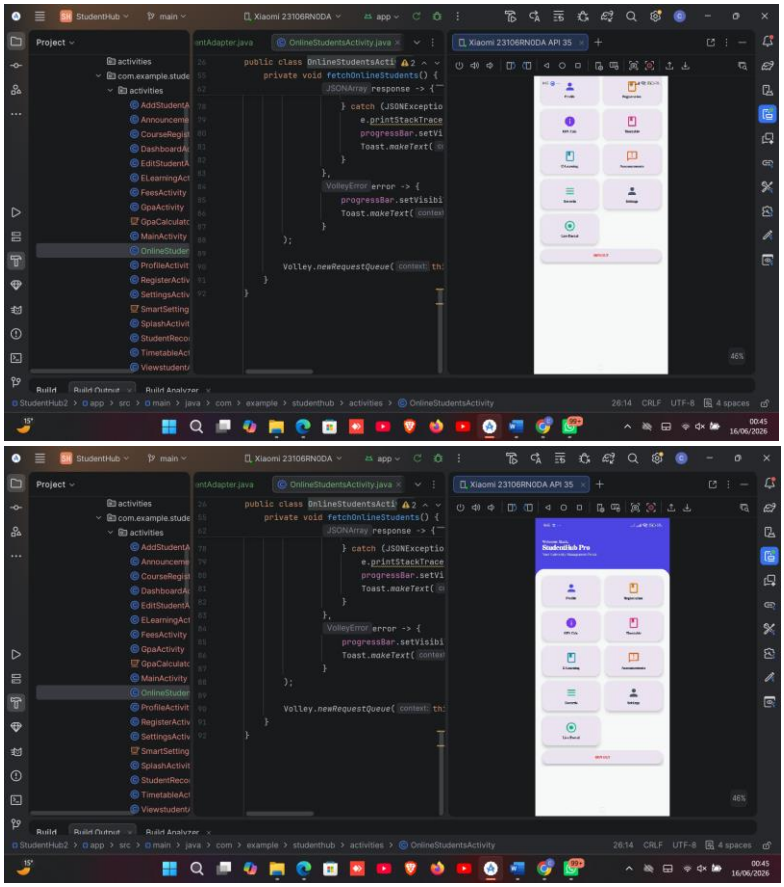


Figure 37Figure 45. RecyclerView displaying records retrieved from API.

TOOLS USED

- Android Studio
- Java
- XML
- Volley Networking Library
- REST API
- JSON
- RecyclerView
- Android Emulator
- Internet Connectivity

PERSONAL REFLECTION

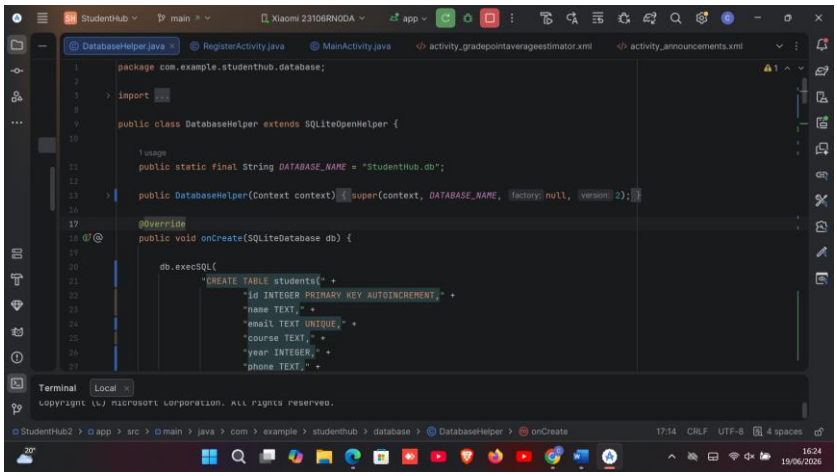
This week enhanced my understanding of mobile networking and API integration. I learned how to retrieve online data, process JSON responses, and display information dynamically. Implementing error handling improved the robustness of the Student Hub application and prepared me for real-world mobile development.

Week 7: Application Architecture

PRACTICAL ACTIVITIES

- Enhanced SQLite database structure
- Added Attendance table with relational design
- Implemented password hashing for secure storage
- Used SharedPreferences for session persistence
- Developed Attendance recording module
- Built Attendance Reports using RecyclerView
- Retrieved and displayed profile data from SQLite
- Tested full data storage and retrieval workflow

Updated Database Structure



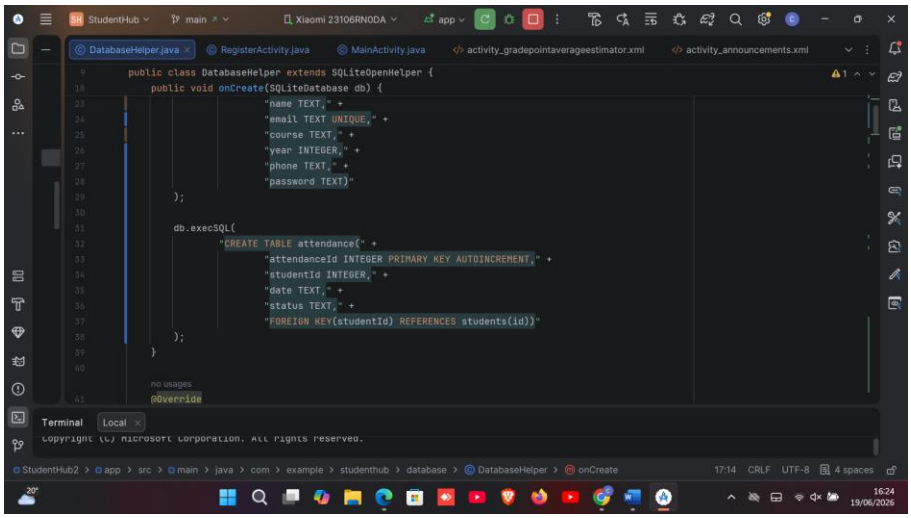
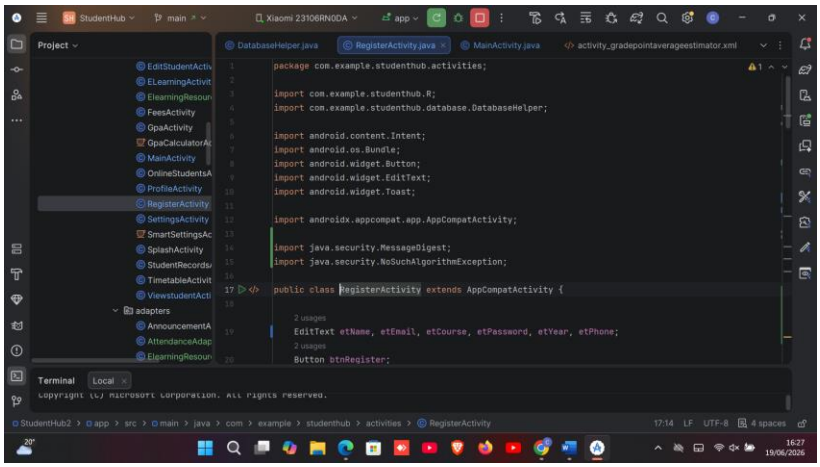
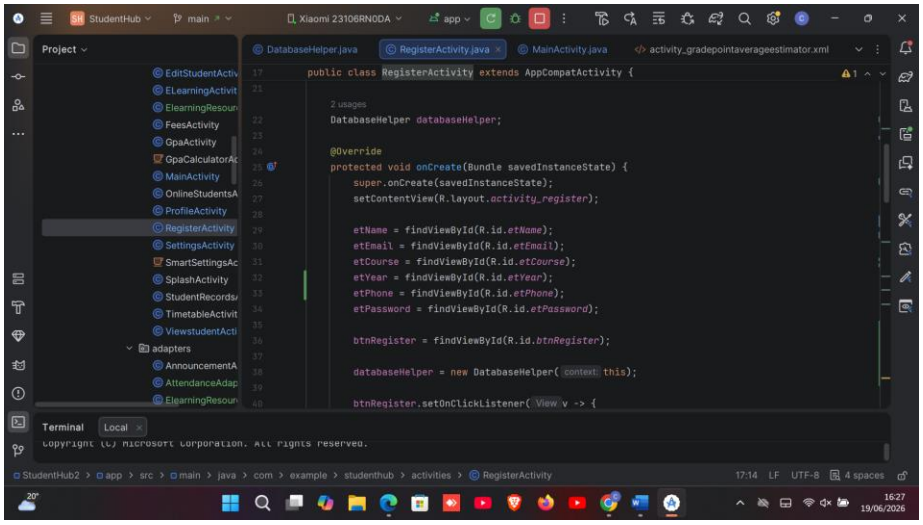
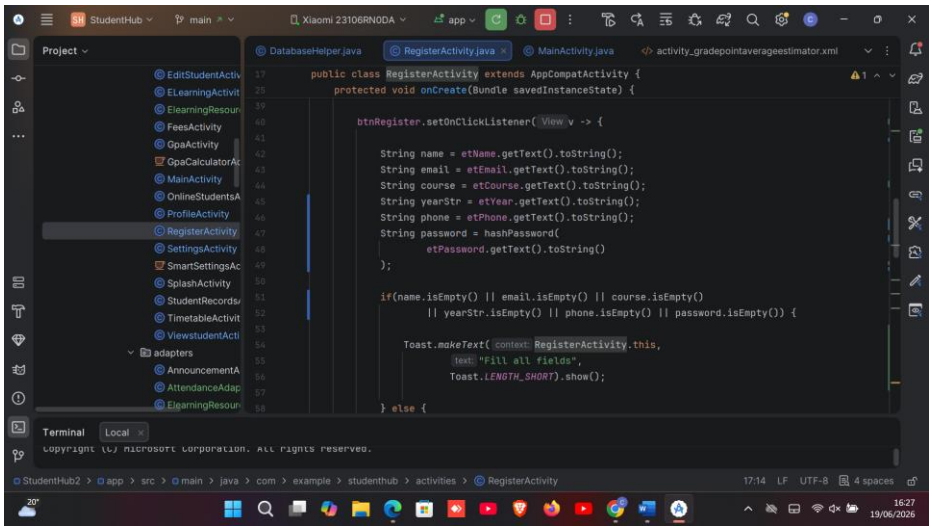


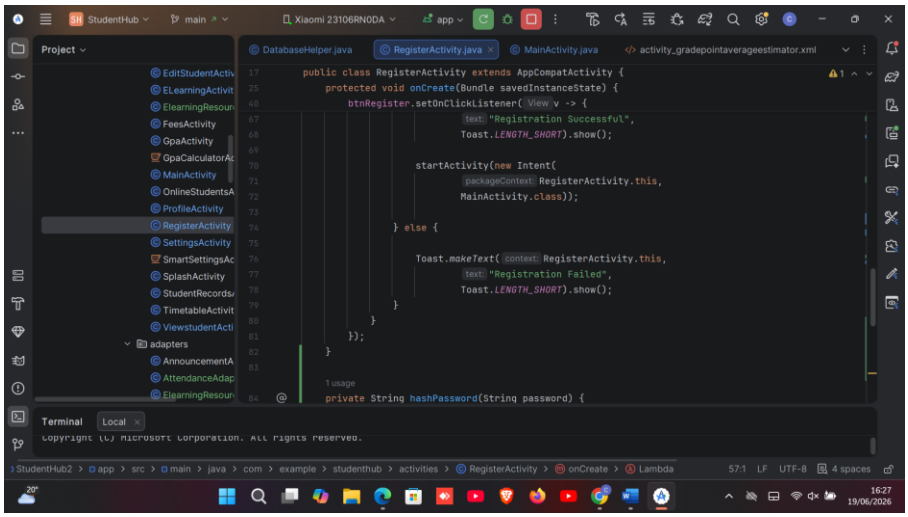
Figure 38.Updated SQLite database structure showing Students and Attendance tables.

Password Hashing Implementation




public class RegisterActivity extends AppCompatActivity {
2 usages
DatabaseHelper databaseHelper;
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_register);
etName = findViewById(R.id.etName);
etEmail = findViewById(R.id.etEmail);
etCourse = findViewById(R.id.etCourse);
etYear = findViewById(R.id.etYear);
etPhone = findViewById(R.id.etPhone);
etPassword = findViewById(R.id.etPassword);
btnRegister = findViewById(R.id.btnRegister);
databaseHelper = new DatabaseHelper(context: this);
btnRegister.setOnClickListener(View v -> {


btnRegister.setOnClickListener(View v -> {
String name = etName.getText().toString();
String email = etEmail.getText().toString();
String course = etCourse.getText().toString();
String yearStr = etYear.getText().toString();
String phone = etPhone.getText().toString();
String password = hashPassword(
etPassword.getText().toString()
);
if(name.isEmpty() || email.isEmpty() || course.isEmpty()
|| yearStr.isEmpty() || phone.isEmpty() || password.isEmpty()) {
Toast.makeText(context: RegisterActivity.this,
text: "Fill all fields",
Toast.LENGTH_SHORT).show();
} else {


text: "Registration Successful",
Toast.LENGTH_SHORT).show();
startActivity(new Intent(
packageContext: RegisterActivity.this,
MainActivity.class));
} else {
Toast.makeText(context: RegisterActivity.this,
text: "Registration Failed",
Toast.LENGTH_SHORT).show();
}
}
});
}
1 usage
private String hashPassword(String password) {

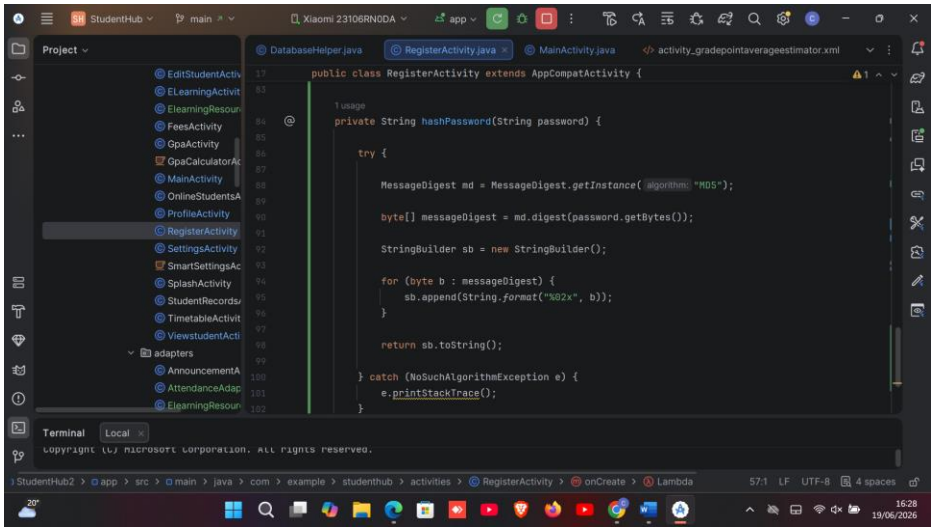
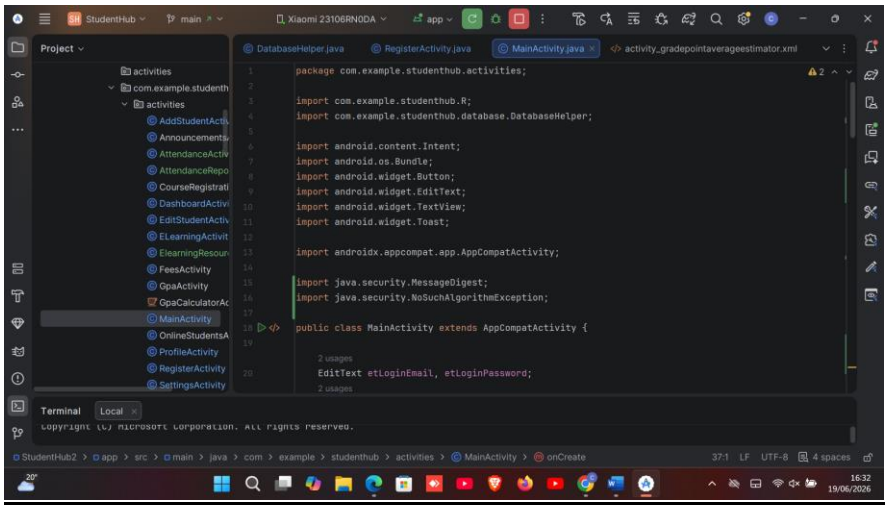
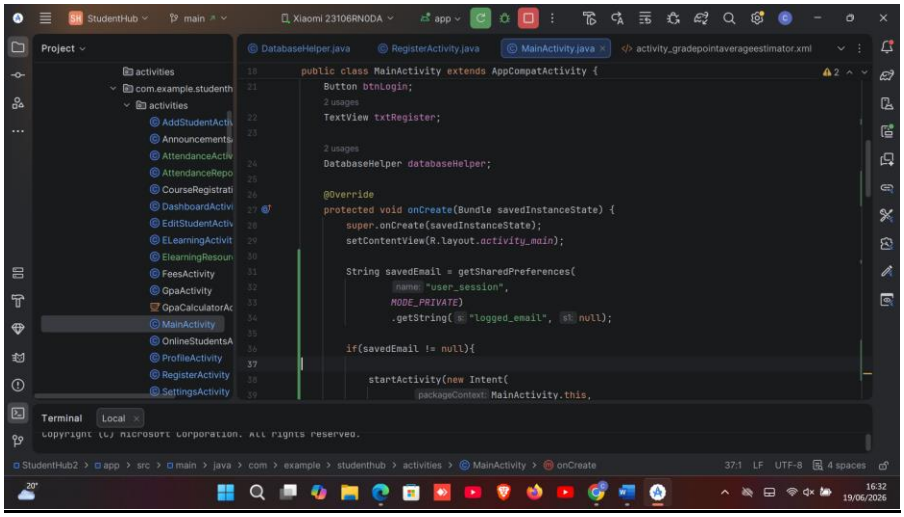
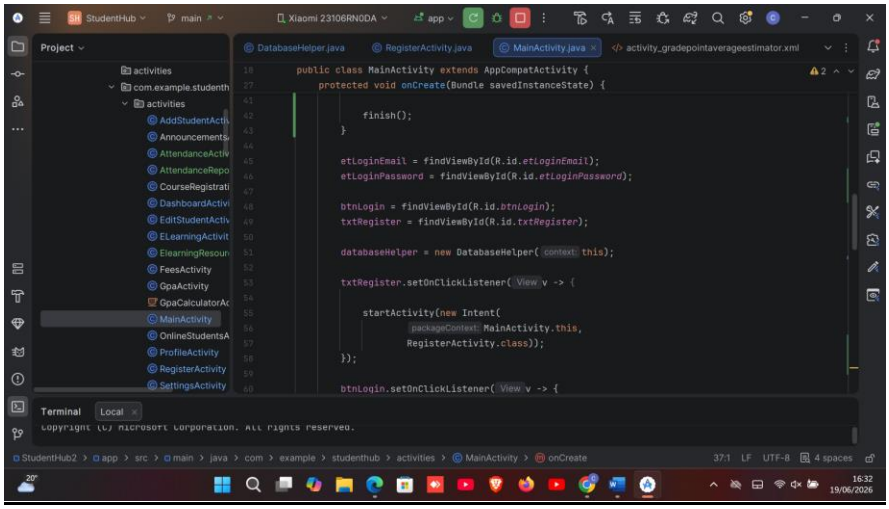


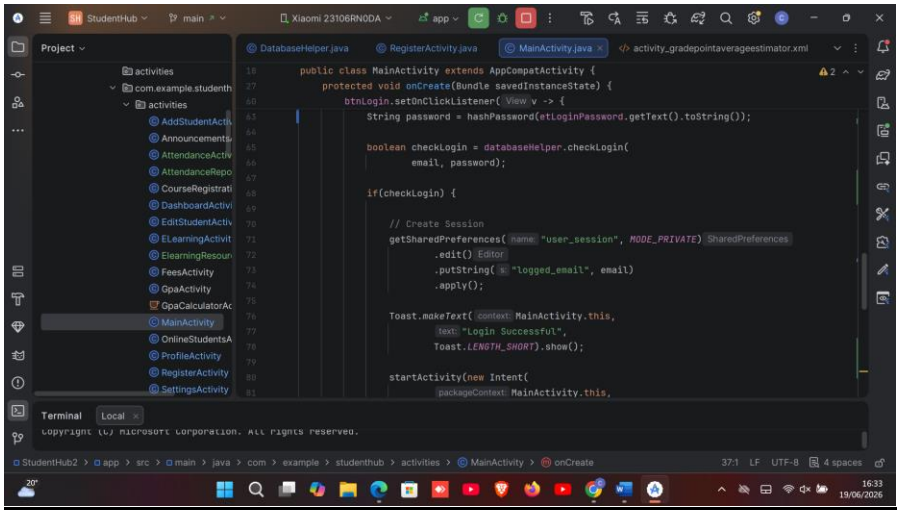
Figure 39Password hashing implementation for secure user authentication.

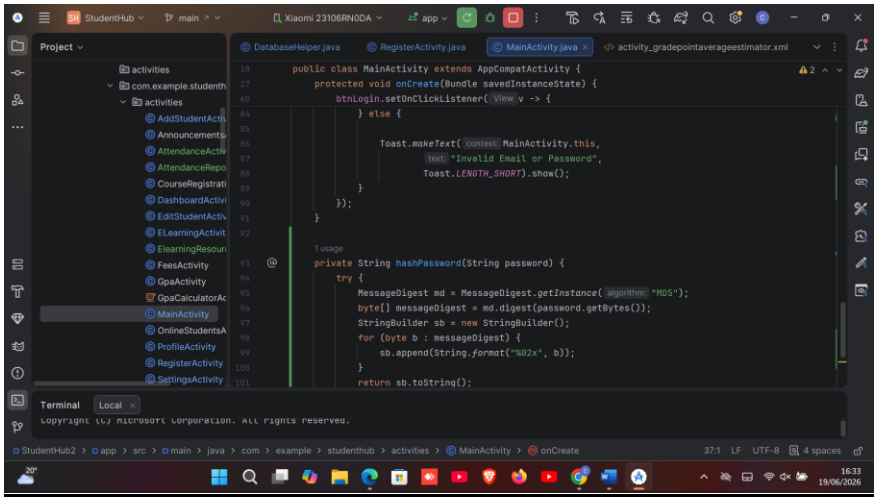
Shared Preferences Session






public class MainActivity extends AppCompatActivity {
protected void onCreate(Bundle savedInstanceState) {
finish();
}
etLoginEmail = findViewById(R.id.etLoginEmail);
etLoginPassword = findViewById(R.id.etLoginPassword);
btnLogin = findViewById(R.id.btnLogin);
txtRegister = findViewById(R.id.txtRegister);
databaseHelper = new DatabaseHelper(context: this);
txtRegister.setOnClickListener(View v -> {
startActivity(new Intent(
packageContext: MainActivity.this,
RegisterActivity.class));
});
btnLogin.setOnClickListener(View v -> {
Terminal
Local
Copyright (C) Microsoft Corporation. All rights reserved.


public class MainActivity extends AppCompatActivity {
protected void onCreate(Bundle savedInstanceState) {
btnLogin.setOnClickListener(View v -> {
String password = hashPassword(etLoginPassword.getText().toString());
boolean checkLogin = databaseHelper.checkLogin(
email, password);
if(checkLogin) {
// Create Session
getSharedPreferences(name: "user_session", MODE_PRIVATE) SharedPreferences
.edit() Editor
.putString(s: "logged_email", email)
.apply();
Toast.makeText(context: MainActivity.this,
text: "Login Successful",
Toast.LENGTH_SHORT).show();
startActivity(new Intent(
packageContext: MainActivity.this,
Terminal
Local
Copyright (C) Microsoft Corporation. All rights reserved.


public class MainActivity extends AppCompatActivity {
protected void onCreate(Bundle savedInstanceState) {
btnLogin.setOnClickListener(View v -> {
} else {
Toast.makeText(context: MainActivity.this,
text: "Invalid Email or Password",
Toast.LENGTH_SHORT).show();
}
});
}
1 usage
private String hashPassword(String password) {
try {
MessageDigest md = MessageDigest.getInstance(algorithm: "MD5");
byte[] messageDigest = md.digest(password.getBytes());
StringBuilder sb = new StringBuilder();
for (byte b : messageDigest) {
sb.append(String.format("%02x", b));
}
return sb.toString();
Terminal
Local
Copyright (C) Microsoft Corporation. All rights reserved.

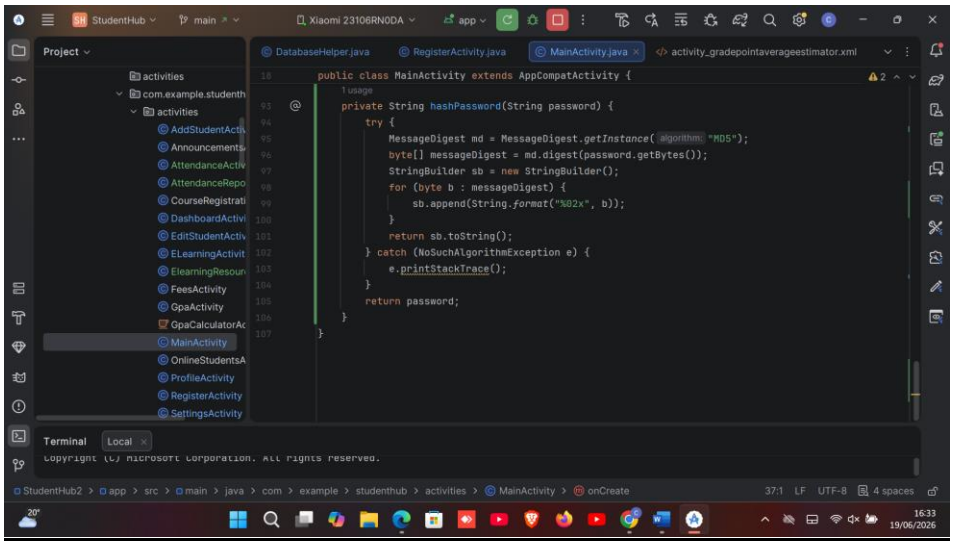


Figure 40Session management using Shared Preferences.

Attendance Module

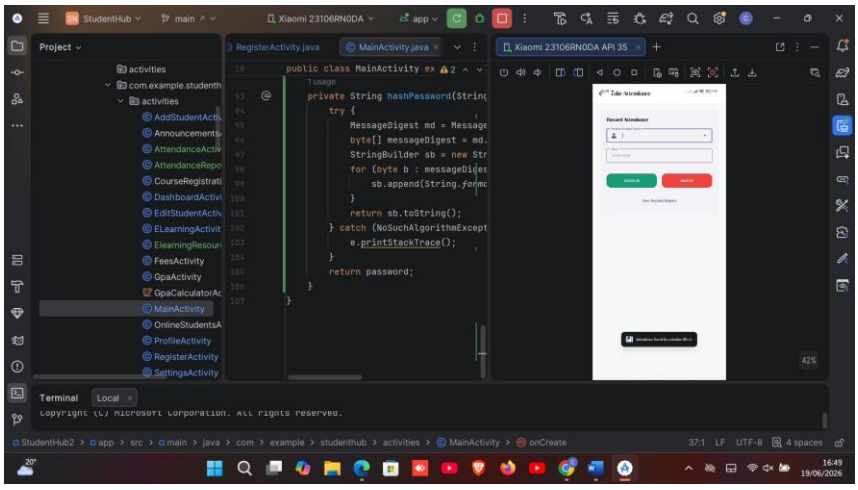
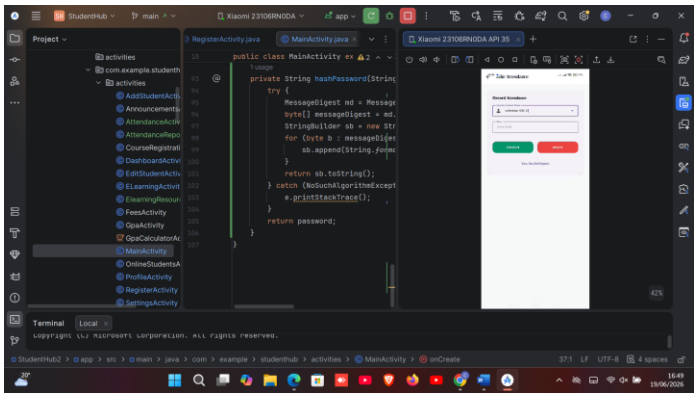


Figure 41.Attendance recording interface

Attendance written & Saved



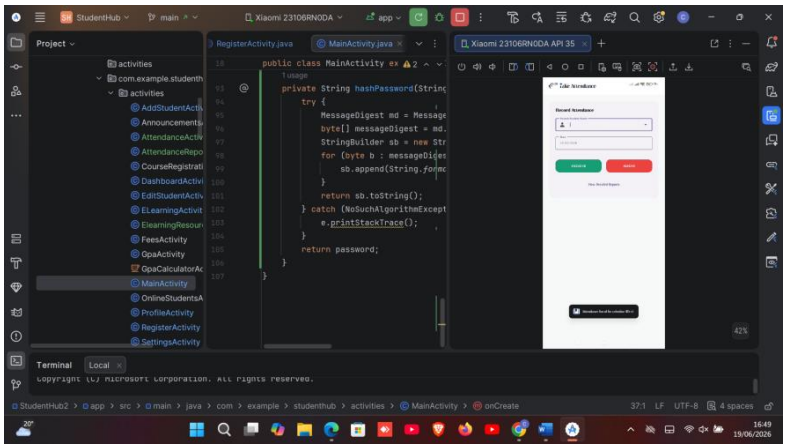


Figure 42Successful attendance record storage

Attendance Reports

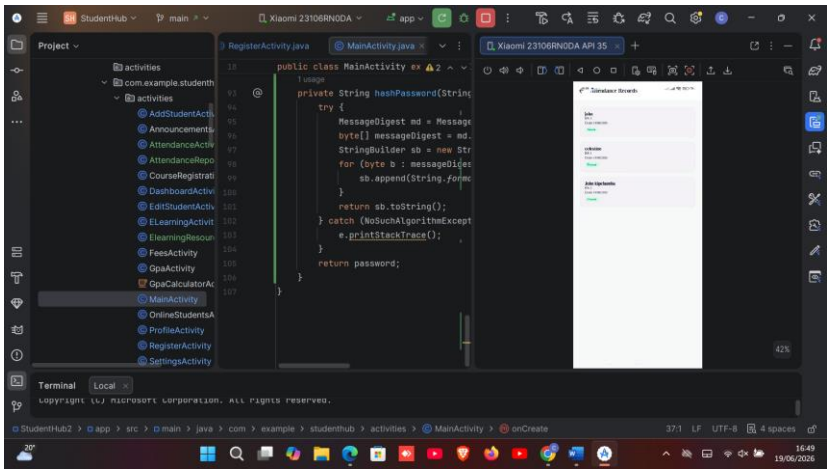


Figure 43Attendance report displaying stored attendance records.

Profile Data Retrieval

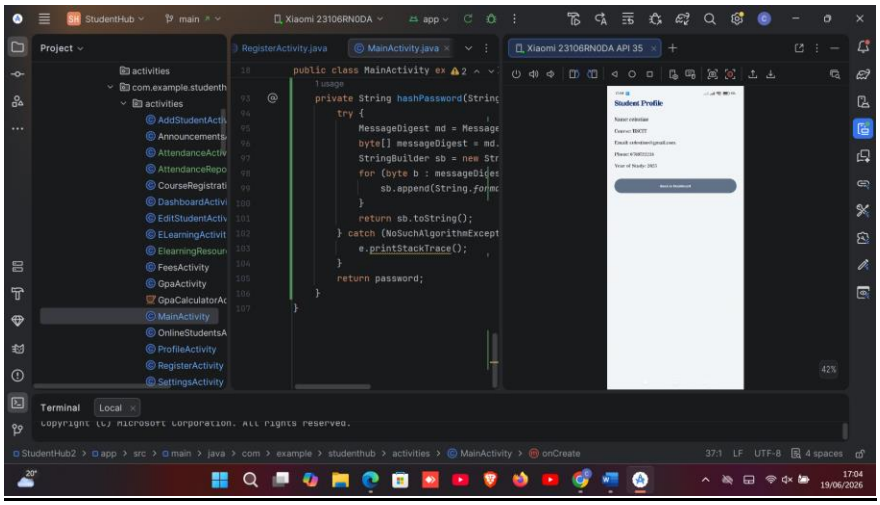


Figure 44Student profile information retrieved from persistent storage.

TOOLS USED

- Android Studio
- Java
- XML
- SQLite Database
- Shared Preferences
- RecyclerView
- Android Emulator
- Material Design Components

PERSONAL REFLECTION

Week 7 improved my understanding of data storage and database integration. I implemented attendance management, session persistence, password security, and database relationships, enhancing the functionality, security, and reliability of the Student Hub application.

Week 8: Project Planning

PRACTICAL ACTIVITIES

1. Created reusable event-handling classes using Object-Oriented Programming.
2. Implemented keyboard input validation and event handling.
3. Developed touch gesture handling for tap, long press, and swipe events.
4. Added event logging for user interactions.
5. Integrated event handling into the Student Hub application.
6. Tested keyboard and touch gesture functionality.

Events Package Structure

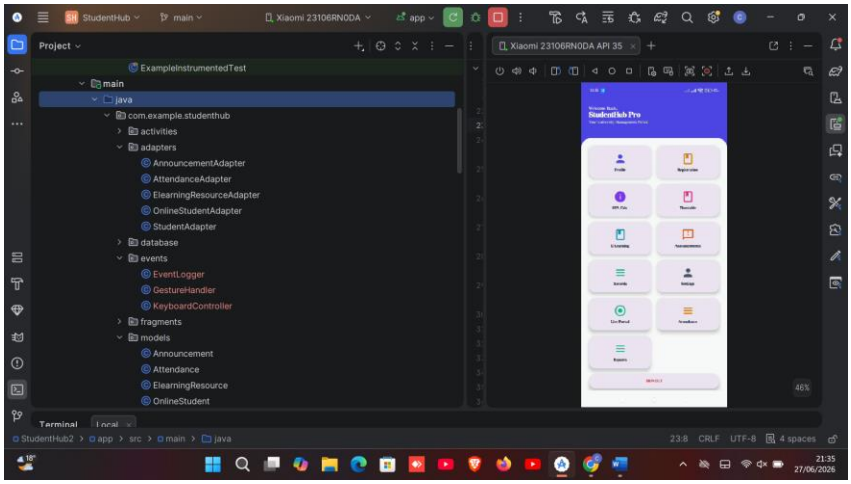


Figure 45Event handling package showing reusable OOP classes.

KeyboardController.java

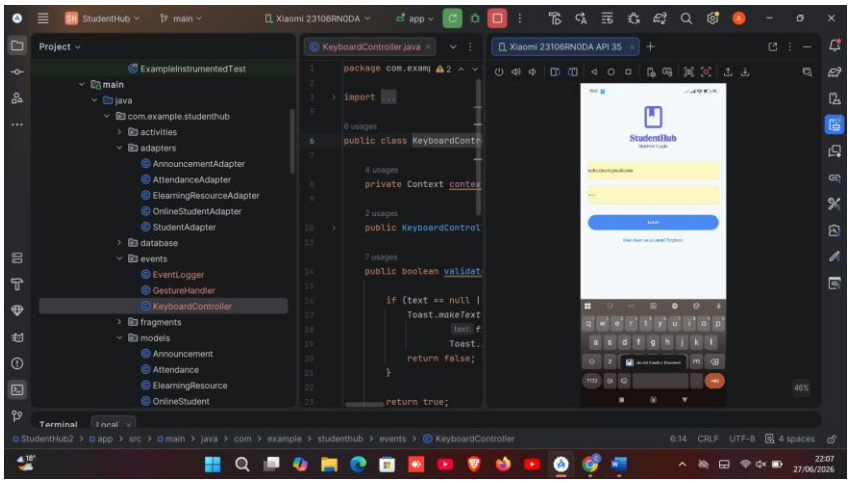
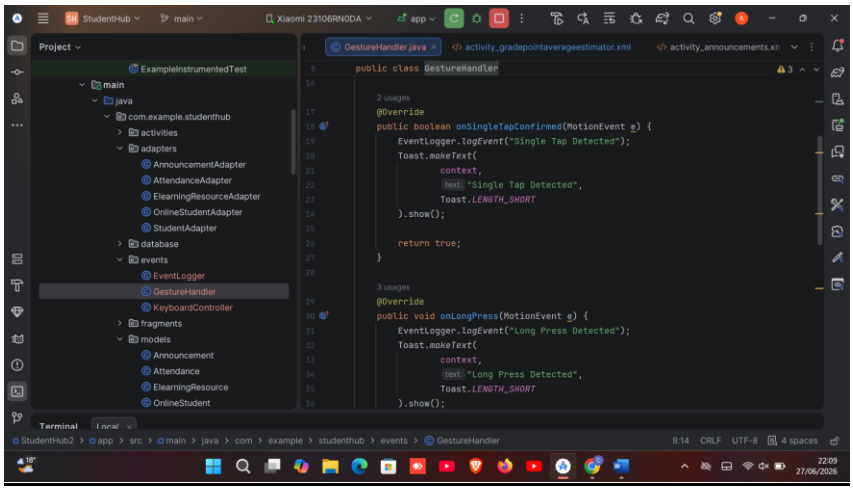


Figure 46Keyboard input handling and validation class.

GestureHandler.java



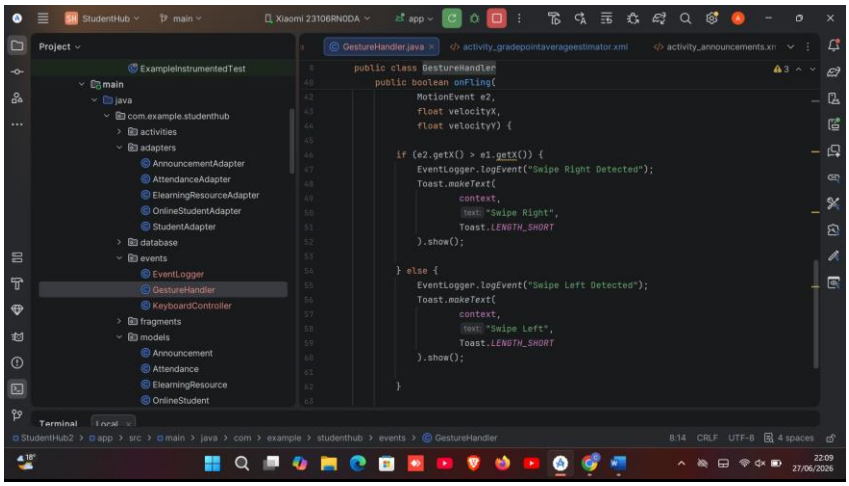


Figure 47Touch gesture handling implementation.

EventLogger.java

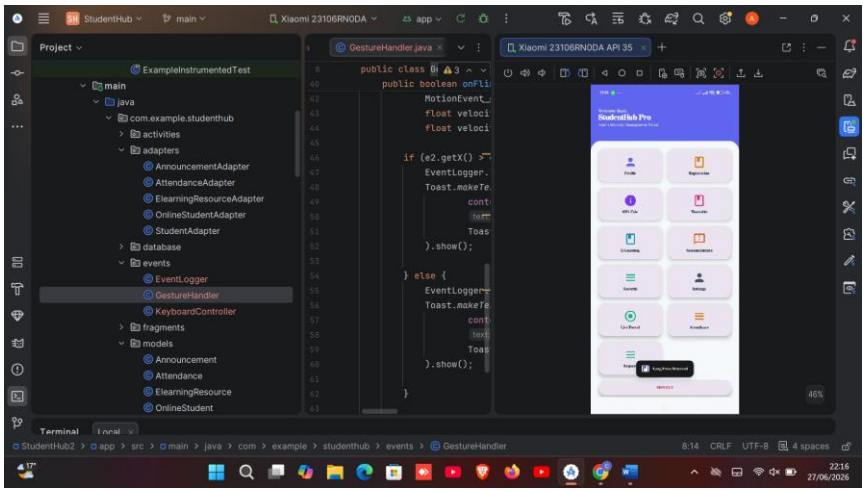
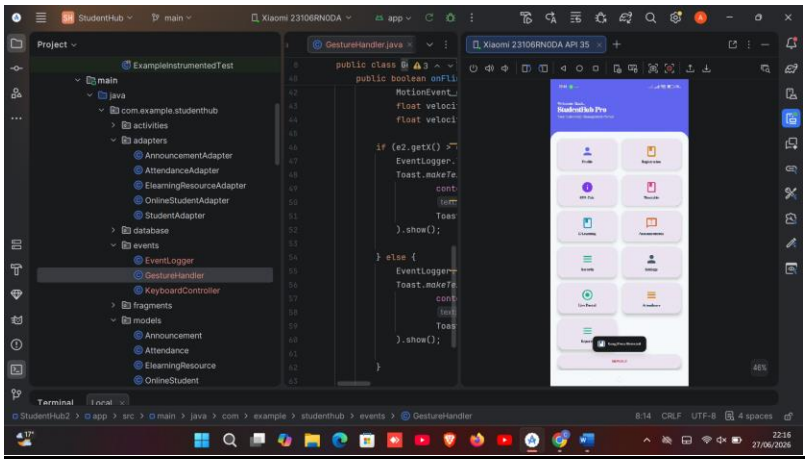


Figure 48Event logging implementation.

Swipe gesture



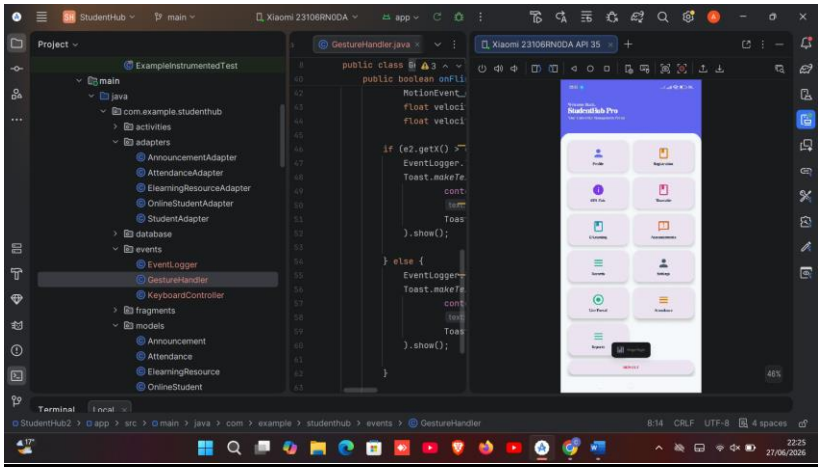
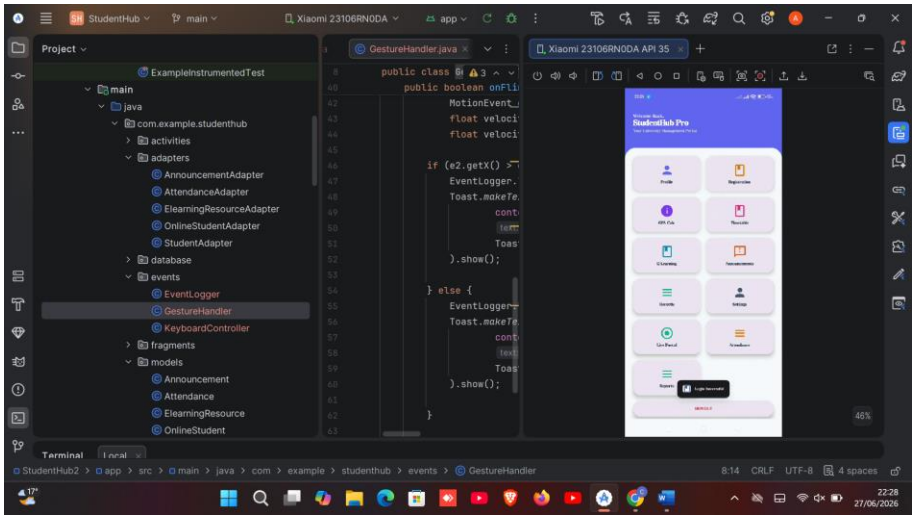


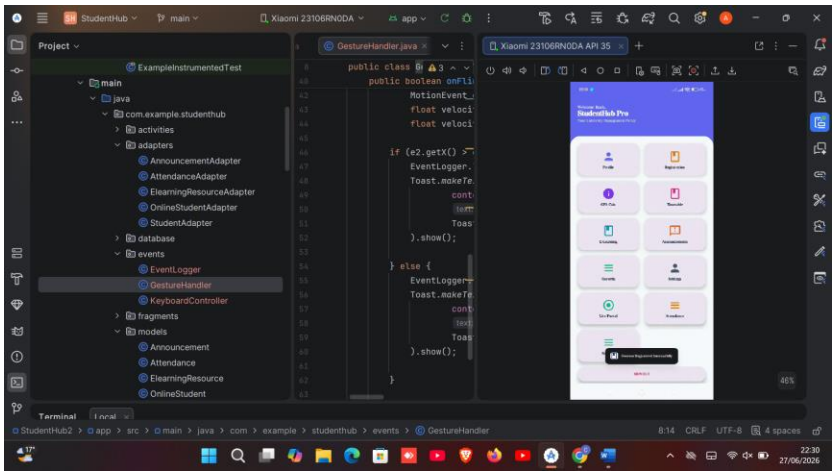
Figure 49swipe gesture from left to right

Event Logger

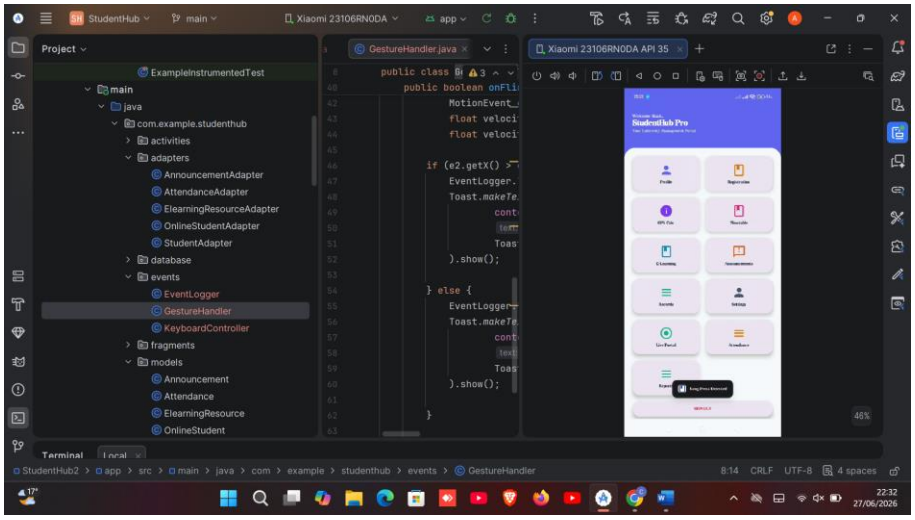
login



Register



longpress



TOOLS USED

- Android Studio
- Java
- XML
- Android SDK
- Gesture Detector API
- Android Emulator
- Logcat

- **Material Design Components**

PERSONAL REFLECTION

Week 8 strengthened my understanding of event-driven programming and object-oriented design. I implemented keyboard input validation, touch gesture handling, and event logging, making the Student Hub application more interactive, modular, and user-friendly.

Week 9: Device Features and Sensors

Practical Activities

- Integrated camera functionality into the application.
- Retrieved user location using GPS.
- Stored campus building information in SQLite.
- Displayed saved building records using RecyclerView.
- Tested runtime permissions for camera and location access.

Campus Explorer Dashboard Card

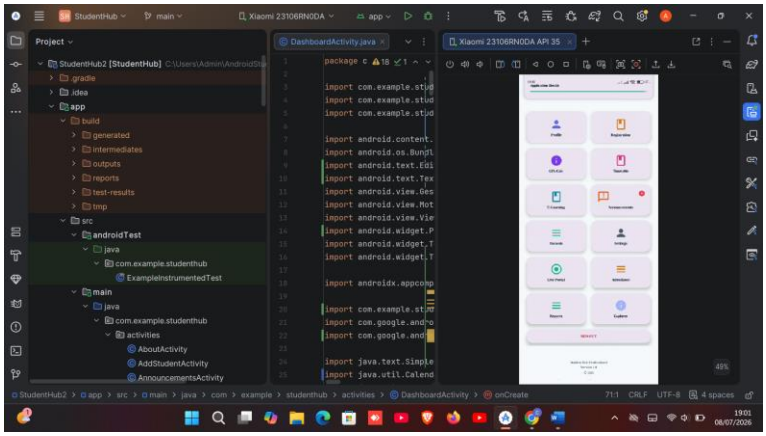


Figure 50new Campus Explorer card.

Campus Explorer Screen

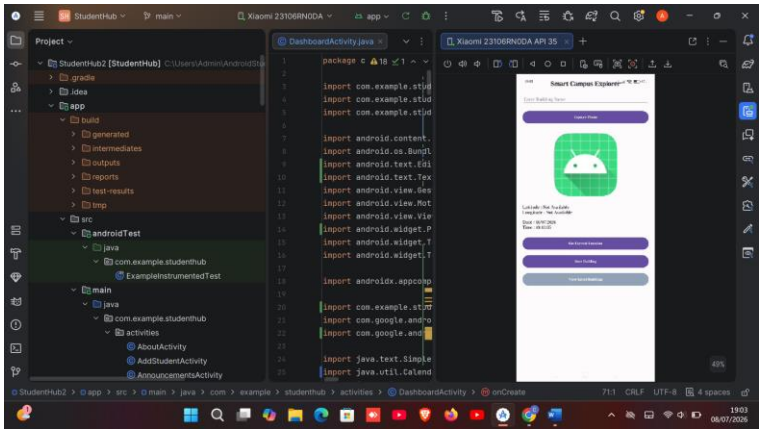


Figure 51 EXPLORER SCREEN

Press Capture Photo AND GPS WORKING

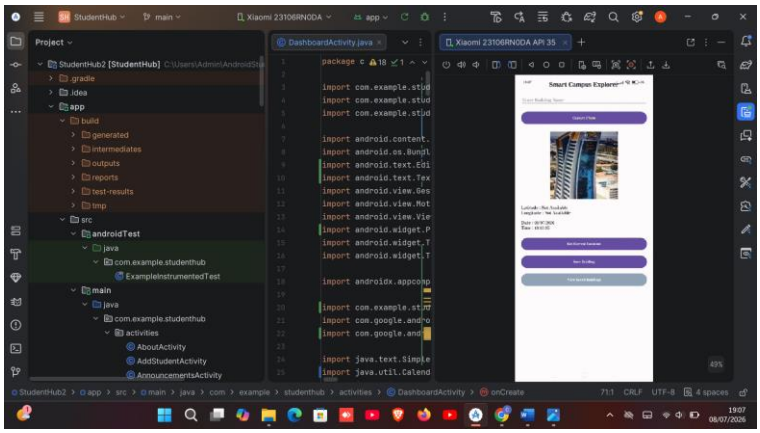


Figure 52Press Capture Photo AND GPS WORKING

Saved Building

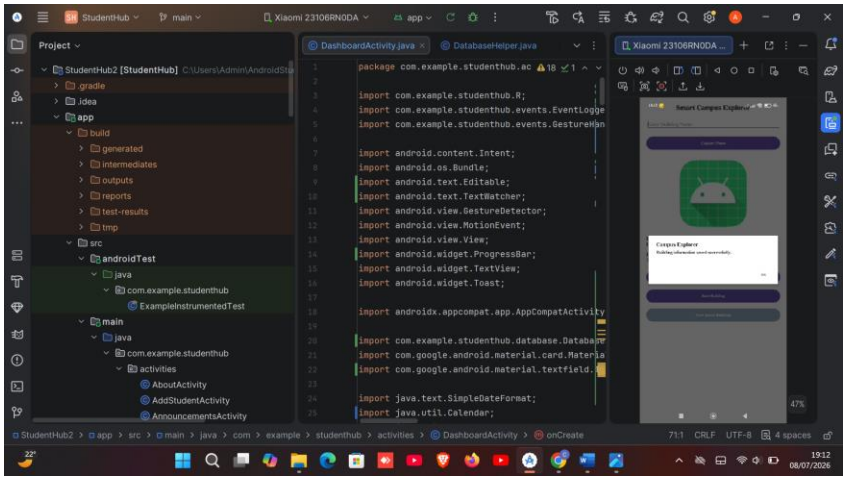


Figure 53Screenshot showing the success message.

Tools Used

- Android Studio
- Java
- XML
- SQLite Database
- Android Camera
- GPS Location Services
- RecyclerView

- Android Emulator

Personal Reflection

Week 9 enhanced my practical skills in integrating Android device features. I successfully implemented camera and GPS functionality, managed runtime permissions, stored location data in SQLite, and improved my understanding of hardware integration within mobile applications.

Week 10: Notifications and Background Services

Practical Activities

- Integrated all application modules into one system.
- Enhanced the dashboard with professional navigation.
- Improved user interface and user experience.
- Implemented input validation and error handling.
- Tested application functionality and resolved bugs.
- Verified database, camera, GPS, and attendance modules.

Splash Screen

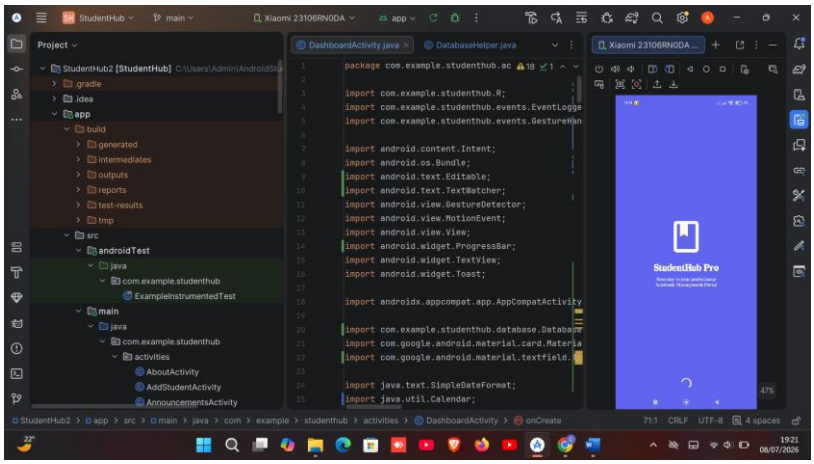
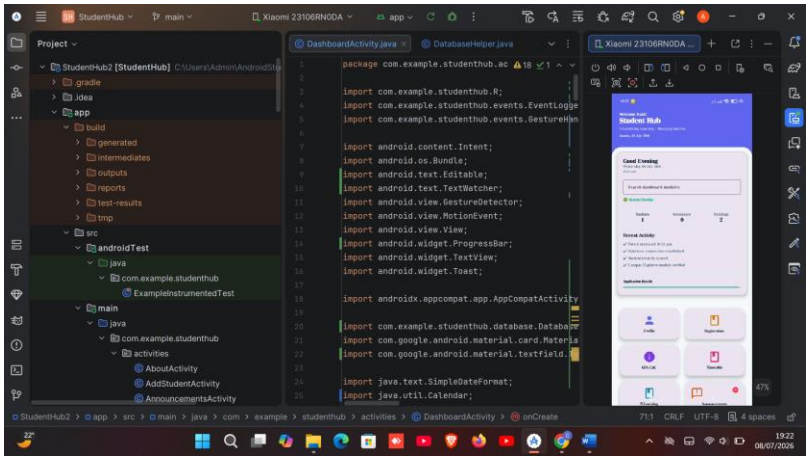


Figure 54 CAPTURED SPLASH SCREEN BEFORE OPENING

Professional Dashboard



Live Dashboard Statistics

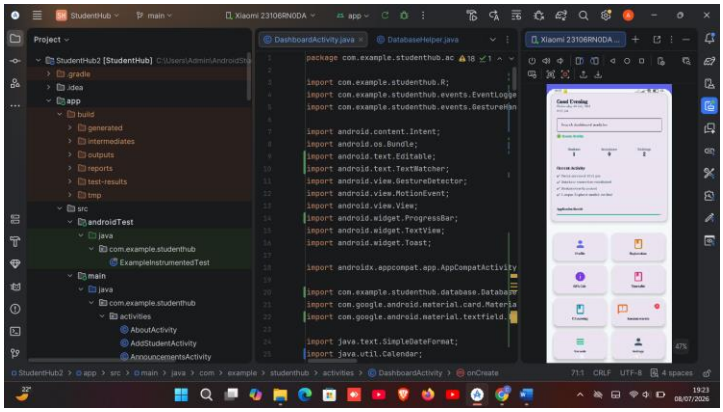
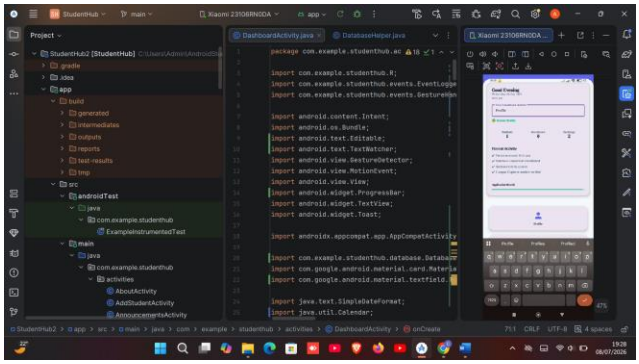


Figure 55Live Dashboard Statistics

Search Function



Tools Used

- Android Studio
- Java
- XML
- SQLite Database
- Recycler View

- Material Design Components
- Android Emulator
- Logcat

Personal Reflection

Week 10 strengthened my understanding of integrating, testing, and refining a complete Android application. I successfully connected all modules, improved the dashboard, enhanced navigation and validation, fixed bugs through testing, and gained valuable experience in developing a reliable, user-friendly, and professional mobile application.

Week 11: Mobile Security

Same as week 1

Week 12: Testing and Debugging

Same as week 1

Week 13: Application Deployment

Same as week 1

Week 14: Final Project Presentation

Same as week 1

Semester Project Tracking

Project Title

Project Objectives

Project Requirements

Development Milestones

Testing Results

Lessons Learned

Future Improvements

End of Semester Reflection

Summary of Skills Acquired

Most Interesting Topic

Major Challenges Faced

How Challenges Were Overcome

Professional Growth Achieved

Overall Course Experience

Final Approval

Student Name: _____

Student Signature: _____

Date: _____

Lecturer Name: _____

Lecturer Remarks: _____

Date: _____